

ENGINEERING DESIGN DOCUMENT

StayVeo

Student Accommodation Marketplace

Version 2.0 · **Date** July 2026

Author CTO Office — StayVeo Engineering

Classification Internal — Confidential

Status Approved for Development

StayVeo — Engineering Design Document (EDD)

Version: 2.0
Date: July 20, 2026
Author: CTO Office — StayVeo Engineering
Classification: Internal — Confidential
Status: APPROVED FOR DEVELOPMENT

Table of Contents

1. Executive Summary	13. API Design
2. Business Context & Product Vision	14. Sequence Diagrams
3. System Architecture	15. Class Diagram (UML)
4. Domain-Driven Design (DDD) — Bounded Contexts	16. Flowcharts
5. Database Design	17. Security Architecture
6. Entity Relationship Diagram (ERD)	18. Scalability & Performance
7. Booking Flow — End to End	19. Edge Cases & Failure Handling
8. Booking State Machine	20. Tech Stack
9. Inventory Management System	21. Folder Structure
10. Offline Inventory Management	22. Deployment Strategy
11. Payment Architecture	23. Monitoring & Observability
12. Notification Engine	24. Future Improvements

1. Executive Summary

StayVeo is a student accommodation marketplace connecting students with PG (Paying Guest), hostel, flat, and room providers. The platform enables students to discover, compare, and reserve accommodations online — while property owners manage inventory, bookings, and occupancy from a unified dashboard.

Core Problem: Students struggle to find verified PGs near their colleges. PG owners rely on word-of-mouth and miss potential tenants. There is no reliable inventory system that handles both online and offline bookings without friction.

Key Engineering Challenges:

CHALLENGE	COMPLEXITY
Dual-channel inventory (online + offline bookings)	HIGH
Race conditions on concurrent bed reservations	HIGH
Partial payment model (reservation fee only at booking)	MEDIUM
Multi-provider notification routing	MEDIUM
Booking state machine with 11+ states	HIGH
Offline booking reconciliation for walk-in customers	HIGH

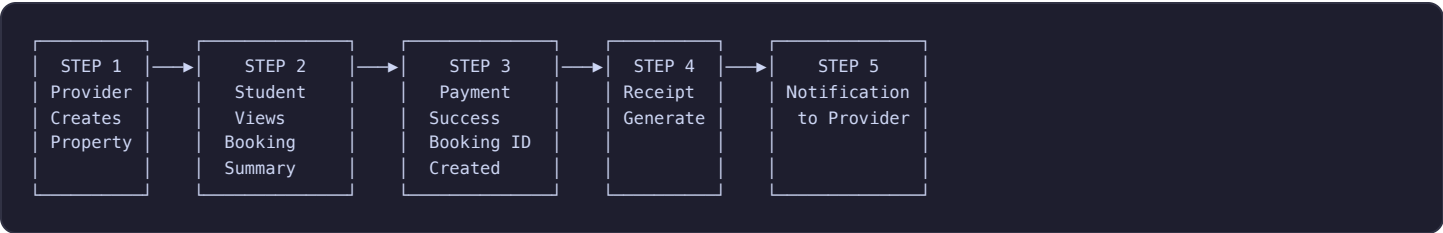
Architecture Decision: We adopt a **Modular Monolith** for MVP (discussed in §3.3) with clear domain boundaries, event-driven communication between modules, and a migration path to microservices at scale.

2. Business Context & Product Vision

2.1 Core Actors

ACTOR	ROLE
Student	Discovers properties, views booking summary, pays reservation fee + platform fee, receives receipt
Provider	Creates properties, defines room types/pricing/charges, manages inventory, receives booking notifications
Admin	Monitors platform health, resolves disputes, manages providers, handles refunds

2.2 Booking Flow Overview



2.3 Property Creation — Provider Fields

The provider creates a property with the following data model:

FIELD	TYPE	REQUIRED	NOTES
PG Name	String	YES	Display name
Property Type	Enum	YES	PG / Hostel / Flat / Room
Address	String	YES	Full address
Location (lat/lng)	Decimal	YES	Geo-coordinates
Room Types	JSON Array	YES	Array of room type objects
Number of Beds (per room type)	Integer	YES	Total beds per room type
Base Rent	Decimal	YES	Monthly rent
Security Deposit	Decimal	YES	Refundable deposit
Reservation Fee	Decimal	YES	Paid at booking
Platform Fee	Decimal	SYSTEM	Calculated by platform
Amenities	String[]	YES	WiFi, AC, Geyser, etc.
Images	String[]	YES	Supabase Storage URLs
Refund Policy	Enum	YES	Full / Partial / None
Minimum Stay	Integer	YES	In months

Custom Charges System

The owner can define custom charges dynamically. This is modeled as a flexible key-value structure:

CHARGE NAME	AMOUNT	FREQUENCY	MANDATORY
Food Charges	₹3,500	Monthly	Optional
Electricity Charges	₹500	Monthly	Mandatory
Water Charges	₹200	Monthly	Mandatory
Maintenance Charges	₹400	Monthly	Mandatory
Parking Charges	₹800	Monthly	Optional
Laundry Charges	₹600	Monthly	Optional
Owner-defined custom	Variable	Variable	Variable

Database Design for Custom Charges:

```
CREATE TABLE property_charges (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  property_id UUID NOT NULL REFERENCES properties(id) ON DELETE CASCADE,  
  charge_name VARCHAR(100) NOT NULL,           -- "Food", "Parking", "Gym Fee"  
  charge_slug VARCHAR(100) NOT NULL,           -- "food", "parking", "gym_fee"  
  amount      DECIMAL(10,2) NOT NULL,  
  frequency   VARCHAR(20) DEFAULT 'monthly',  -- monthly | quarterly | one_time  
  is_mandatory BOOLEAN DEFAULT false,  
  is_system   BOOLEAN DEFAULT false,         -- true for platform-defined charges  
  display_order INT DEFAULT 0,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  
  UNIQUE(property_id, charge_slug)  
);
```

The owner dashboard provides:

- Pre-populated system charges (Food, Electricity, Water, Maintenance, Parking, Laundry)
- An "+ Add Custom Charge" button where the owner types a name and amount
- Each charge has a toggle for mandatory/optional

2.4 Booking Summary — Student View

When a student opens a property, they see:

BOOKING SUMMARY

PG Name: Green Valley PG
Room Type: Double Sharing
Rent: ₹8,000 /month
Security Deposit: ₹8,000
Reservation Fee: ₹2,000
Platform Fee: ₹499

ADDITIONAL CHARGES

Food: ₹3,500 /month
Electricity: ₹500 /month
Maintenance: ₹400 /month
Parking: ₹800 /month (optional)

TOTAL MONTHLY COST: ₹13,200

YOU PAY NOW

Reservation Fee: ₹2,000
Platform Fee: ₹499

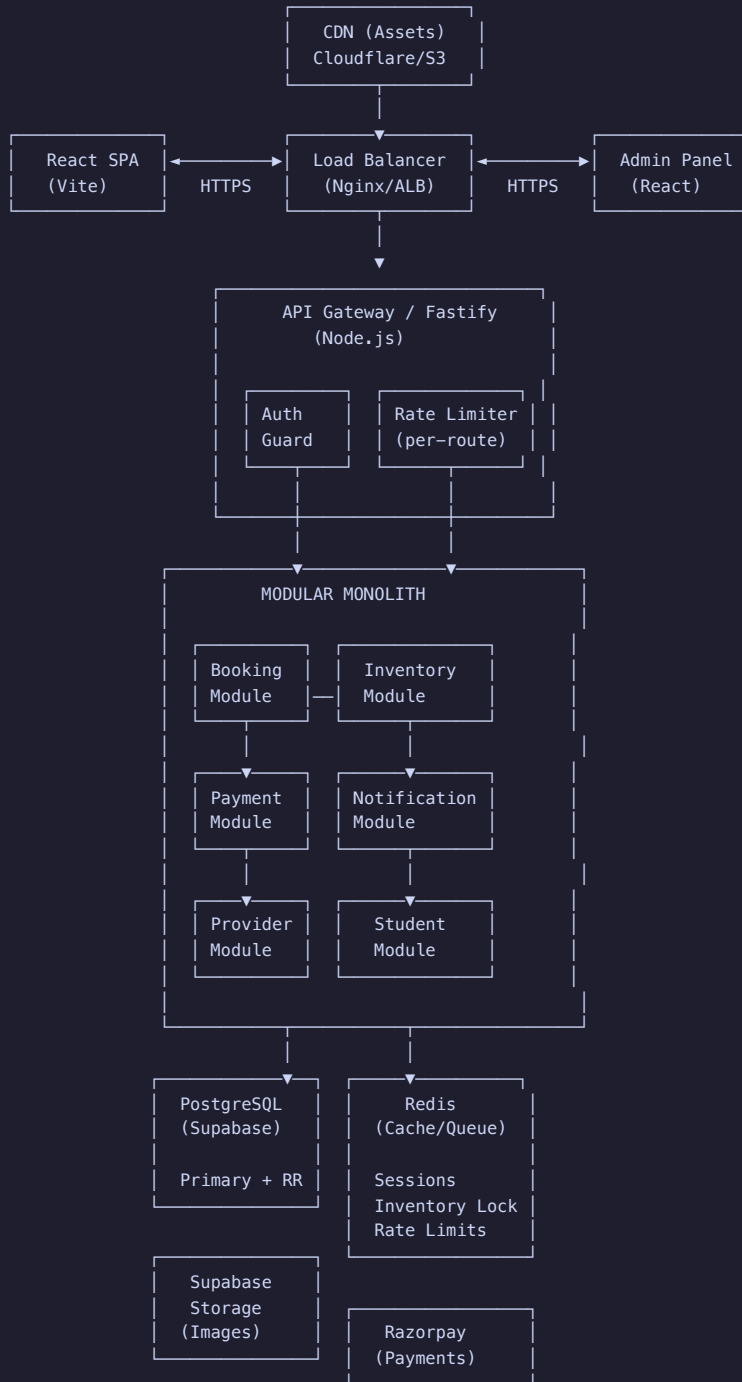
TOTAL PAYABLE: ₹2,499

Note: Rent & deposits are paid directly
at the property during move-in.

[RESERVE NOW – ₹2,499]

3. System Architecture

3.1 High-Level Architecture



3.2 Architecture Layers (Clean Architecture)



3.3 Microservices vs Modular Monolith — Decision

CRITERIA	MICROSERVICES	MODULAR MONOLITH
Development speed	SLOW — service boundaries, inter-service comms	FAST — single codebase
Operational complexity	HIGH — K8s, service mesh, distributed tracing	LOW — single deploy
Team size needed	5-10 engineers minimum	2-3 engineers
Data consistency	Eventual (saga patterns)	Strong (DB transactions)
Cost	HIGH — multiple containers, networking	LOW — single server
Debugging	HARD — distributed logs	EASY — single process
Scaling story	Independent scaling per service	Vertical + horizontal app scaling
Migration to microservices	Already there	Clean split along module boundaries

Decision: MODULAR MONOLITH for MVP

Rationale: StayVeo is a seed-stage startup. The team is small. The modular monolith gives us:

- Strong data consistency (critical for inventory/payments)
- Fast iteration speed
- Single deployment pipeline
- Clear module boundaries that can be extracted to microservices later
- Shared database with schema-level isolation

Migration Path: Each module (Booking, Inventory, Payment, Notification) is designed with clear interfaces. When any single module becomes a bottleneck, it can be extracted into its own service with:

- Its own database (schema split)
- Message queue for inter-service communication
- Independent scaling

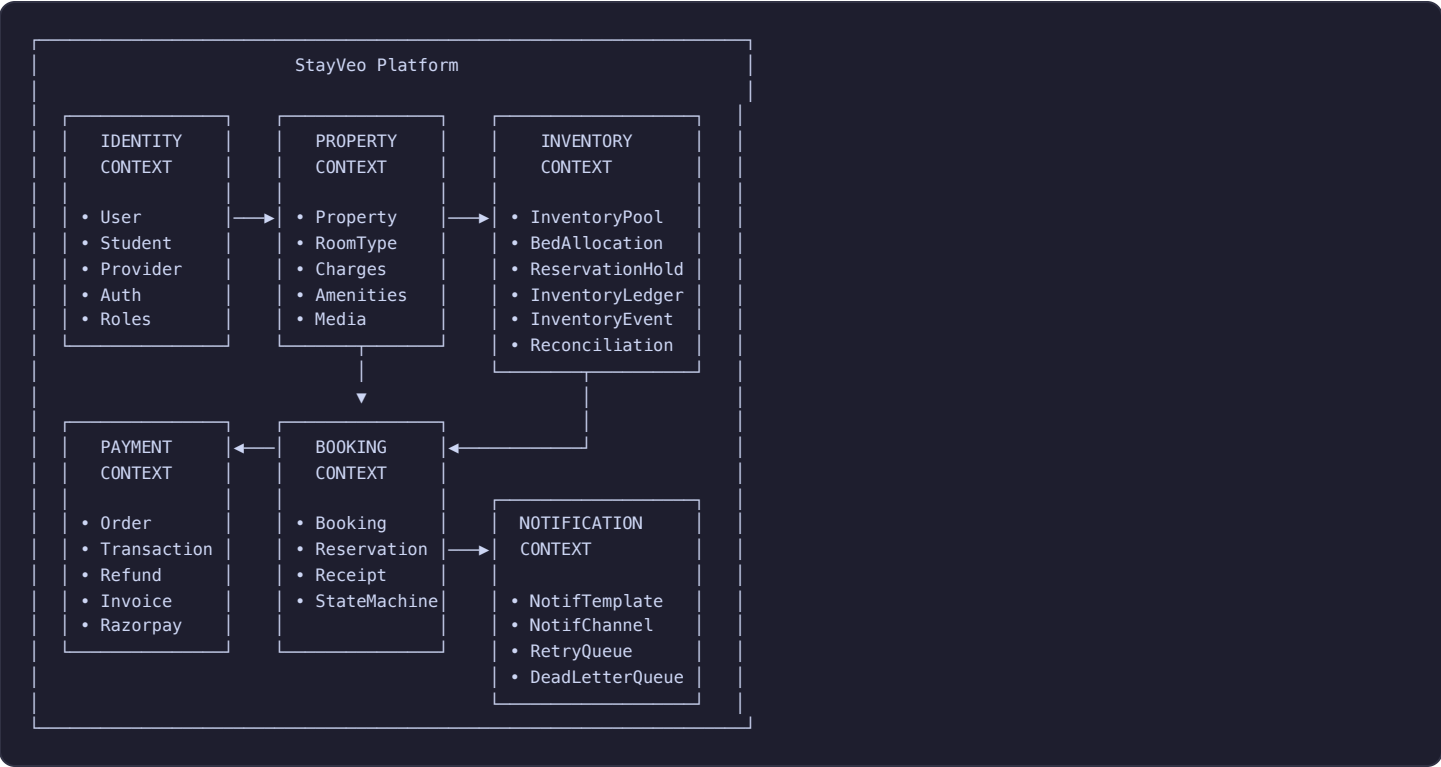
3.4 SOLID Principles Application

PRINCIPLE	APPLICATION IN STAYVEO
Single Responsibility	Each module owns exactly one domain. BookingService handles bookings. InventoryService handles inventory. They never overlap.

PRINCIPLE	APPLICATION IN STAYVEO
Open/Closed	New property types (co-living, apartment) are added via configuration, not code changes. Charge types are extensible via the <code>property_charges</code> table.
Liskov Substitution	All payment processors (Razorpay, Stripe future) implement <code>PaymentGateway</code> interface. Swapping is transparent.
Interface Segregation	<code>BookingReader</code> and <code>BookingWriter</code> are separate interfaces. Student dashboard only uses <code>BookingReader</code> .
Dependency Inversion	Business logic depends on abstractions (Repository interfaces), never on Prisma directly. Infrastructure layer implements these interfaces.

4. Domain-Driven Design (DDD) — Bounded Contexts

4.1 Bounded Context Map



4.2 Aggregate Roots

AGGREGATE	ROOT ENTITY	OWNED ENTITIES
Property	<code>Property</code>	<code>RoomType</code> , <code>PropertyCharge</code> , <code>PropertyImage</code> , <code>PropertyAmenity</code>
Inventory	<code>InventoryPool</code>	<code>BedAllocation</code> , <code>InventoryLedger</code> , <code>InventorySnapshot</code>
Booking	<code>Booking</code>	<code>BookingPayment</code> , <code>BookingReceipt</code> , <code>BookingStateHistory</code>
Payment	<code>PaymentOrder</code>	<code>PaymentTransaction</code> , <code>Refund</code> , <code>Invoice</code>
Notification	<code>NotificationBatch</code>	<code>Notification</code> , <code>NotificationLog</code>

5. Database Design

5.1 Complete Schema — Production Grade

5.1.1 Users Table

```
CREATE TABLE users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  phone_number VARCHAR(15) UNIQUE NOT NULL,  
  email       VARCHAR(255),  
  role        user_role NOT NULL,          -- 'student' | 'provider'  
  college_id  UUID REFERENCES colleges(id) ON DELETE SET NULL,  
  college_name VARCHAR(255),  
  is_active   BOOLEAN DEFAULT true,  
  is_verified BOOLEAN DEFAULT false,  
  last_login_at TIMESTAMPTZ,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_users_phone ON users(phone_number);  
CREATE INDEX idx_users_role ON users(role);  
CREATE INDEX idx_users_college ON users(college_id);
```

5.1.2 Providers Table

```
CREATE TABLE providers (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  name        VARCHAR(200) NOT NULL,  
  business_name VARCHAR(300) NOT NULL,  
  phone_number VARCHAR(15) NOT NULL,  
  email       VARCHAR(255),  
  location    VARCHAR(500),  
  latitude    DECIMAL(9,6) NOT NULL,  
  longitude   DECIMAL(9,6) NOT NULL,  
  service_radius_km DECIMAL(5,2),  
  is_verified  BOOLEAN DEFAULT false,  
  verification_date TIMESTAMPTZ,  
  avg_rating   DECIMAL(2,1) DEFAULT 0,  
  total_reviews INT DEFAULT 0,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_providers_user ON providers(user_id);  
CREATE INDEX idx_providers_location ON providers USING GIST (  
  ll_to_earth(latitude, longitude)  
);
```

5.1.3 Properties Table

```
CREATE TABLE properties (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  provider_id       UUID NOT NULL REFERENCES providers(id) ON DELETE CASCADE,  
  name              VARCHAR(300) NOT NULL,           -- "Green Valley PG"  
  property_type     property_type NOT NULL,         -- pg | hostel | flat | room  
  description       TEXT,  
  address           TEXT NOT NULL,  
  city              VARCHAR(100) NOT NULL,  
  state             VARCHAR(100),  
  pincode           VARCHAR(10),  
  latitude          DECIMAL(9,6) NOT NULL,  
  longitude         DECIMAL(9,6) NOT NULL,  
  gender_preference VARCHAR(20) DEFAULT 'unisex',  
  
  -- Pricing defaults  
  reservation_fee   DECIMAL(10,2) NOT NULL,  
  platform_fee_pct   DECIMAL(4,2) DEFAULT 5.00,      -- platform fee percentage  
  security_deposit   DECIMAL(10,2),  
  refund_policy      refund_policy NOT NULL DEFAULT 'partial',  
  minimum_stay_months INT DEFAULT 1,  
  
  -- Status  
  is_active         BOOLEAN DEFAULT true,  
  is_verified       BOOLEAN DEFAULT false,  
  is_featured       BOOLEAN DEFAULT false,  
  
  -- SEO/Search  
  slug              VARCHAR(400) UNIQUE,  
  
  created_at        TIMESTAMPTZ DEFAULT NOW(),  
  updated_at        TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_properties_provider ON properties(provider_id);  
CREATE INDEX idx_properties_city ON properties(city);  
CREATE INDEX idx_properties_type ON properties(property_type);  
CREATE INDEX idx_properties_active ON properties(is_active, is_verified);  
CREATE INDEX idx_properties_geo ON properties USING GIST (  
  ll_to_earth(latitude, longitude)  
);  
-- Partition by city for scale (future)
```

5.1.4 Property Rooms Table

```
CREATE TABLE property_rooms (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  property_id       UUID NOT NULL REFERENCES properties(id) ON DELETE CASCADE,  
  room_type         VARCHAR(100) NOT NULL,          -- "Single" | "Double" | "Triple"  
  title             VARCHAR(200) NOT NULL,          -- "Single AC Room"  
  description        TEXT,  
  floor             INT,  
  
  -- Pricing  
  rent              DECIMAL(10,2) NOT NULL,         -- monthly rent  
  security_deposit   DECIMAL(10,2),  
  
  -- Inventory  
  total_beds        INT NOT NULL DEFAULT 1,  
  available_beds     INT NOT NULL DEFAULT 1,  
  reserved_beds      INT NOT NULL DEFAULT 0,  
  occupied_beds      INT NOT NULL DEFAULT 0,  
  offline_beds       INT NOT NULL DEFAULT 0,         -- walk-in occupancy  
  blocked_beds       INT NOT NULL DEFAULT 0,         -- maintenance  
  
  -- Features  
  amenities          TEXT[] DEFAULT '{}',  
  images             TEXT[] DEFAULT '{}',  
  gender_preference   VARCHAR(20) DEFAULT 'unisex',  
  
  -- Status  
  status             room_status DEFAULT 'active',  
  is_active          BOOLEAN DEFAULT true,  
  
  created_at         TIMESTAMPTZ DEFAULT NOW(),  
  updated_at         TIMESTAMPTZ DEFAULT NOW(),  
  
  -- Inventory integrity constraint  
  CONSTRAINT chk_bed_count CHECK (  
    available_beds >= 0 AND  
    reserved_beds >= 0 AND  
    occupied_beds >= 0 AND  
    offline_beds >= 0 AND  
    blocked_beds >= 0 AND  
    available_beds + reserved_beds + occupied_beds + offline_beds + blocked_beds = total_beds  
  )  
);  
  
CREATE INDEX idx_rooms_property ON property_rooms(property_id);  
CREATE INDEX idx_rooms_available ON property_rooms(available_beds) WHERE is_active = true;  
CREATE INDEX idx_rooms_type ON property_rooms(room_type);
```

5.1.5 Property Charges Table

```
CREATE TABLE property_charges (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  property_id       UUID NOT NULL REFERENCES properties(id) ON DELETE CASCADE,  
  charge_name       VARCHAR(100) NOT NULL,  
  charge_slug       VARCHAR(100) NOT NULL,  
  amount            DECIMAL(10,2) NOT NULL,  
  frequency         charge_frequency DEFAULT 'monthly', -- monthly | quarterly | one_time  
  is_mandatory      BOOLEAN DEFAULT false,  
  is_system         BOOLEAN DEFAULT false,  
  description        TEXT,  
  display_order     INT DEFAULT 0,  
  created_at        TIMESTAMPTZ DEFAULT NOW(),  
  
  UNIQUE(property_id, charge_slug)  
);  
  
CREATE INDEX idx_charges_property ON property_charges(property_id);
```

5.1.6 Bookings Table

```
CREATE TABLE bookings (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  booking_id        VARCHAR(20) UNIQUE NOT NULL,      -- "STV-2026-000001"
  student_id        UUID NOT NULL REFERENCES users(id),
  provider_id        UUID NOT NULL REFERENCES providers(id),
  property_id        UUID NOT NULL REFERENCES properties(id),
  room_id            UUID NOT NULL REFERENCES property_rooms(id),

  -- Amounts
  reservation_amount DECIMAL(10,2) NOT NULL,
  platform_fee        DECIMAL(10,2) NOT NULL,
  total_paid           DECIMAL(10,2) NOT NULL,          -- reservation + platform
  pending_amount       DECIMAL(10,2) NOT NULL,          -- rent + deposit (paid at move-in)
  monthly_rent         DECIMAL(10,2) NOT NULL,
  security_deposit     DECIMAL(10,2),

  -- Status
  status              booking_status NOT NULL DEFAULT 'payment_pending',
  reservation_status   reservation_status DEFAULT 'held',

  -- Dates
  move_in_date         DATE NOT NULL,
  move_out_date        DATE,
  check_in_at          TIMESTAMPTZ,
  check_out_at         TIMESTAMPTZ,

  -- Payment
  payment_status       payment_status DEFAULT 'pending',
  payment_gateway_id   VARCHAR(100),                  -- Razorpay order/payment ID
  payment_method       VARCHAR(50),

  -- Student info snapshot
  student_name         VARCHAR(200),
  student_phone        VARCHAR(15),
  student_email        VARCHAR(255),

  -- Metadata
  notes                TEXT,
  cancellation_reason   TEXT,
  cancelled_by         VARCHAR(20),                  -- 'student' | 'provider' | 'admin'
  refund_amount        DECIMAL(10,2),
  refund_status        VARCHAR(20),

  created_at           TIMESTAMPTZ DEFAULT NOW(),
  updated_at           TIMESTAMPTZ DEFAULT NOW(),
  expires_at           TIMESTAMPTZ                  -- reservation expiry
);

CREATE INDEX idx_bookings_student ON bookings(student_id);
CREATE INDEX idx_bookings_provider ON bookings(provider_id);
CREATE INDEX idx_bookings_property ON bookings(property_id);
CREATE INDEX idx_bookings_room ON bookings(room_id);
CREATE INDEX idx_bookings_status ON bookings(status);
CREATE INDEX idx_bookings_booking_id ON bookings(booking_id);
CREATE INDEX idx_bookings_date ON bookings(created_at DESC);
CREATE INDEX idx_bookings_movein ON bookings(move_in_date);

-- Partition by created_at (monthly) for scale
-- CREATE TABLE bookings_2026_07 PARTITION OF bookings
--   FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
```

5.1.7 Booking Payments Table

```
CREATE TABLE booking_payments (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  booking_id        UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,

  -- Razorpay fields
  razorpay_order_id  VARCHAR(100) NOT NULL,
  razorpay_payment_id VARCHAR(100),
  razorpay_signature VARCHAR(500),

  -- Amount
  amount            DECIMAL(10,2) NOT NULL,
  currency           VARCHAR(3) DEFAULT 'INR',

  -- Breakdown
  reservation_fee    DECIMAL(10,2) NOT NULL,
  platform_fee        DECIMAL(10,2) NOT NULL,
  gst_amount          DECIMAL(10,2) DEFAULT 0,

  -- Status
  status             payment_status NOT NULL DEFAULT 'pending',
  failure_reason      TEXT,

  -- Metadata
  payment_method      VARCHAR(50),          -- upi | card | netbanking | wallet
  bank_name           VARCHAR(100),
  ip_address           VARCHAR(45),
  user_agent          TEXT,

  -- Webhook
  webhook_received_at TIMESTAMPTZ,
  webhook_event_id     VARCHAR(100),

  created_at          TIMESTAMPTZ DEFAULT NOW(),
  updated_at          TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_bpayments_booking ON booking_payments(booking_id);
CREATE INDEX idx_bpayments_razorpay ON booking_payments(razorpay_order_id);
CREATE INDEX idx_bpayments_status ON booking_payments(status);
```

5.1.8 Invoices Table

```
CREATE TABLE invoices (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  invoice_number    VARCHAR(30) UNIQUE NOT NULL,    -- "STV-INV-2026-000001"  
  booking_id        UUID NOT NULL REFERENCES bookings(id),  
  student_id        UUID NOT NULL REFERENCES users(id),  
  provider_id       UUID NOT NULL REFERENCES providers(id),  
  
  -- Amounts  
  subtotal          DECIMAL(10,2) NOT NULL,  
  gst_amount        DECIMAL(10,2) DEFAULT 0,  
  total_amount      DECIMAL(10,2) NOT NULL,  
  
  -- Breakdown JSON  
  line_items        JSONB NOT NULL,  
  /*  
  [  
    { "name": "Reservation Fee", "amount": 2000 },  
    { "name": "Platform Fee", "amount": 499 },  
    { "name": "GST (18%)", "amount": 89.82 }  
  ]  
  */  
  
  -- Status  
  status            VARCHAR(20) DEFAULT 'generated', -- generated | sent | void  
  
  -- PDF  
  pdf_url           TEXT,  
  qr_code_data      TEXT,                          -- encoded booking verification URL  
  
  created_at        TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_invoices_booking ON invoices(booking_id);  
CREATE INDEX idx_invoices_number ON invoices(invoice_number);
```

5.1.9 Inventory Ledger

```
CREATE TABLE inventory_ledger (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  room_id           UUID NOT NULL REFERENCES property_rooms(id),  
  property_id       UUID NOT NULL REFERENCES properties(id),  
  
  -- Event  
  event_type        inventory_event NOT NULL,  
/*  
  'reservation_hold' -- student pays, bed held  
  'reservation_release' -- hold expired/cancelled  
  'check_in'         -- student moves in  
  'check_out'        -- student moves out  
  'offline_occupy'   -- walk-in booking  
  'offline_vacate'   -- walk-in leaves  
  'block_maintenance' -- maintenance block  
  'unblock_maintenance' -- maintenance complete  
  'manual_adjustment' -- admin/provider manual fix  
  'reconciliation'  -- daily reconciliation  
*/  
  
  -- Delta  
  beds_delta        INT NOT NULL,                -- +1 or -1  
  field_affected     VARCHAR(30) NOT NULL,        -- 'available' | 'reserved' | 'occupied' | 'offline' | 'blocked'  
  
  -- Snapshot after event  
  available_after    INT NOT NULL,  
  reserved_after     INT NOT NULL,  
  occupied_after     INT NOT NULL,  
  offline_after      INT NOT NULL,  
  blocked_after      INT NOT NULL,  
  
  -- Correlation  
  booking_id         UUID REFERENCES bookings(id),  
  performed_by       UUID REFERENCES users(id),  
  source             VARCHAR(30) DEFAULT 'system', -- 'system' | 'provider' | 'admin' | 'cron'  
  
  -- Metadata  
  reason            TEXT,  
  metadata          JSONB,  
  
  created_at         TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_ledger_room ON inventory_ledger(room_id);  
CREATE INDEX idx_ledger_property ON inventory_ledger(property_id);  
CREATE INDEX idx_ledger_event ON inventory_ledger(event_type);  
CREATE INDEX idx_ledger_date ON inventory_ledger(created_at DESC);  
CREATE INDEX idx_ledger_booking ON inventory_ledger(booking_id);
```

5.1.10 Inventory Snapshots

```
CREATE TABLE inventory_snapshots (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  room_id     UUID NOT NULL REFERENCES property_rooms(id),  
  property_id UUID NOT NULL REFERENCES properties(id),  
  
  snapshot_date DATE NOT NULL,  
  snapshot_time TIME DEFAULT NOW()::TIME,  
  
  total_beds    INT NOT NULL,  
  available_beds INT NOT NULL,  
  reserved_beds INT NOT NULL,  
  occupied_beds INT NOT NULL,  
  offline_beds  INT NOT NULL,  
  blocked_beds  INT NOT NULL,  
  
  occupancy_rate DECIMAL(5,2),          -- percentage  
  
  created_at    TIMESTAMPTZ DEFAULT NOW(),  
  
  UNIQUE(room_id, snapshot_date)  
);  
  
CREATE INDEX idx_snapshots_room_date ON inventory_snapshots(room_id, snapshot_date DESC);  
CREATE INDEX idx_snapshots_property ON inventory_snapshots(property_id);
```

5.1.11 Notifications Table

```
CREATE TABLE notifications (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id),  
  
  -- Content  
  title       VARCHAR(200) NOT NULL,  
  message     TEXT NOT NULL,  
  type        notification_type NOT NULL,  
  /*  
    'payment_success' | 'booking_created' | 'booking_cancelled' |  
    'refund_initiated' | 'refund_completed' | 'move_in_reminder' |  
    'move_out_reminder' | 'inventory_low' | 'property_full' |  
    'offline_conflict' | 'new_booking' | 'provider_action_required'  
  */  
  
  -- Delivery  
  channel      notification_channel NOT NULL,      -- 'push' | 'sms' | 'whatsapp' | 'email' | 'in_app'  
  delivery_status VARCHAR(20) DEFAULT 'pending',  -- 'pending' | 'sent' | 'delivered' | 'failed' | 'dlq'  
  
  -- Reference  
  reference_type VARCHAR(50),                      -- 'booking' | 'payment' | 'inventory'  
  reference_id   UUID,  
  
  -- Retry  
  attempt_count  INT DEFAULT 0,  
  max_attempts   INT DEFAULT 3,  
  next_retry_at  TIMESTAMPTZ,  
  last_error     TEXT,  
  
  -- Read status  
  is_read        BOOLEAN DEFAULT false,  
  read_at        TIMESTAMPTZ,  
  
  -- Scheduling  
  scheduled_at   TIMESTAMPTZ,                      -- for future notifications  
  sent_at        TIMESTAMPTZ,  
  
  created_at     TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_notif_user ON notifications(user_id, is_read, created_at DESC);  
CREATE INDEX idx_notif_status ON notifications(delivery_status) WHERE delivery_status IN ('pending', 'failed');  
CREATE INDEX idx_notif_retry ON notifications(next_retry_at) WHERE delivery_status = 'failed';  
CREATE INDEX idx_notif_scheduled ON notifications(scheduled_at) WHERE delivery_status = 'pending';
```

5.1.12 Reservation Ledger

```
CREATE TABLE reservation_ledger (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  booking_id        UUID NOT NULL REFERENCES bookings(id),  
  room_id           UUID NOT NULL REFERENCES property_rooms(id),  
  
  -- Hold details  
  beds_held         INT NOT NULL DEFAULT 1,  
  hold_started_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  hold_expires_at   TIMESTAMPTZ NOT NULL,           -- typically 48-72 hours  
  
  -- Status  
  status            VARCHAR(20) NOT NULL DEFAULT 'active',  
  -- 'active' | 'confirmed' | 'expired' | 'released' | 'cancelled'  
  
  -- Resolution  
  resolved_at       TIMESTAMPTZ,  
  resolved_by       VARCHAR(20),                   -- 'system' | 'provider' | 'student'  
  resolution_reason TEXT,  
  
  created_at        TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_resled_booking ON reservation_ledger(booking_id);  
CREATE INDEX idx_resled_room ON reservation_ledger(room_id);  
CREATE INDEX idx_resled_status ON reservation_ledger(status) WHERE status = 'active';  
CREATE INDEX idx_resled_expiry ON reservation_ledger(hold_expires_at) WHERE status = 'active';
```

5.1.13 Audit Logs

```
CREATE TABLE audit_logs (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id           UUID REFERENCES users(id),  
  
  -- Action  
  action            VARCHAR(100) NOT NULL,          -- 'booking.create' | 'inventory.adjust' | 'payment.refund'  
  entity_type       VARCHAR(50) NOT NULL,           -- 'booking' | 'property' | 'inventory' | 'payment'  
  entity_id         UUID NOT NULL,  
  
  -- Changes  
  old_value         JSONB,  
  new_value         JSONB,  
  
  -- Context  
  ip_address        VARCHAR(45),  
  user_agent        TEXT,  
  session_id        VARCHAR(100),  
  
  created_at        TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_audit_entity ON audit_logs(entity_type, entity_id);  
CREATE INDEX idx_audit_user ON audit_logs(user_id);  
CREATE INDEX idx_audit_action ON audit_logs(action);  
CREATE INDEX idx_audit_date ON audit_logs(created_at DESC);  
  
-- Partition by month for performance
```

5.1.14 Payment Logs

```
CREATE TABLE payment_logs (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  booking_id        UUID REFERENCES bookings(id),  
  payment_id        UUID REFERENCES booking_payments(id),  
  
  -- Event  
  event_type        VARCHAR(50) NOT NULL,  
  -- 'order_created' | 'payment_authorized' | 'payment_captured' |  
  -- 'payment_failed' | 'refund_initiated' | 'refund_processed' |  
  -- 'webhook_received' | 'webhook_verified' | 'webhook_failed'  
  
  -- Razorpay data  
  razorpay_event_id VARCHAR(100),  
  razorpay_payload   JSONB,  
  
  -- Status  
  status            VARCHAR(20),  
  error_message     TEXT,  
  
  created_at        TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_plog_booking ON payment_logs(booking_id);  
CREATE INDEX idx_plog_payment ON payment_logs(payment_id);  
CREATE INDEX idx_plog_event ON payment_logs(event_type);
```

5.1.15 Booking State History

```
CREATE TABLE booking_state_history (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  booking_id        UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,  
  
  from_state        booking_status,  
  to_state          booking_status NOT NULL,  
  
  triggered_by      VARCHAR(20) NOT NULL,          -- 'system' | 'student' | 'provider' | 'admin'  
  trigger_event     VARCHAR(50) NOT NULL,          -- 'payment_received' | 'provider_confirmed' etc.  
  
  metadata          JSONB,  
  created_at        TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_bsh_booking ON booking_state_history(booking_id, created_at DESC);
```

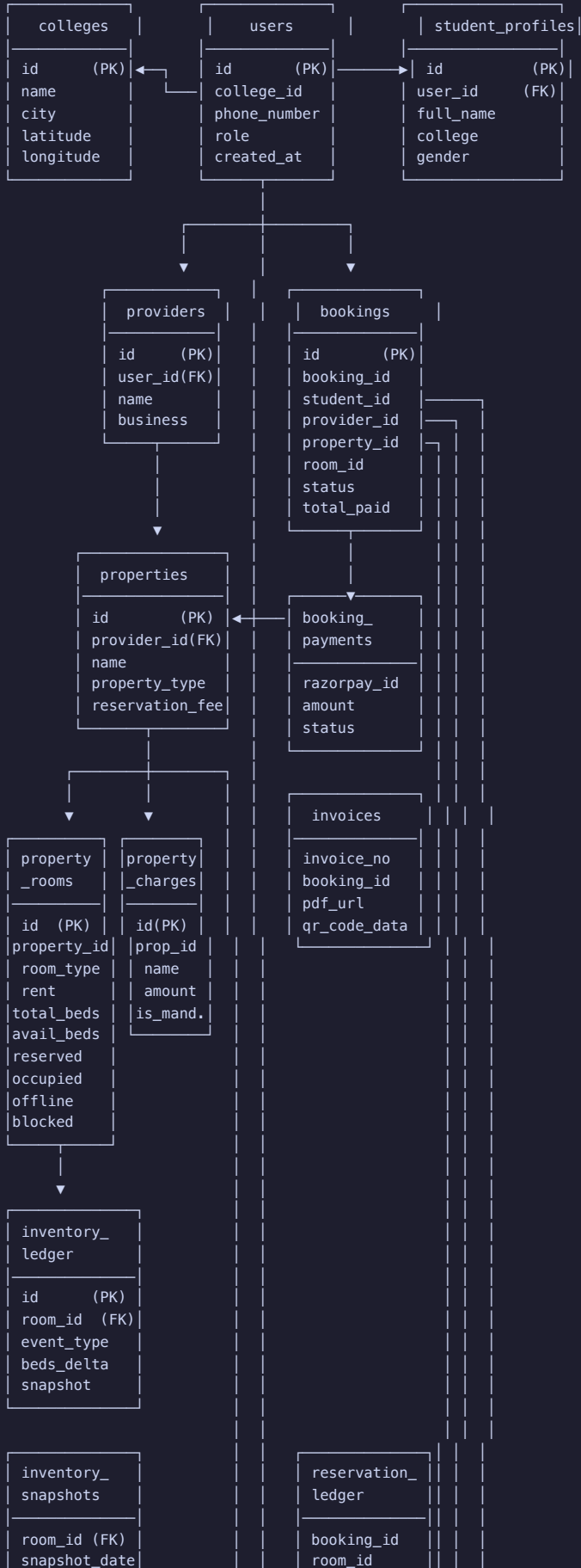
5.2 Enums (PostgreSQL)

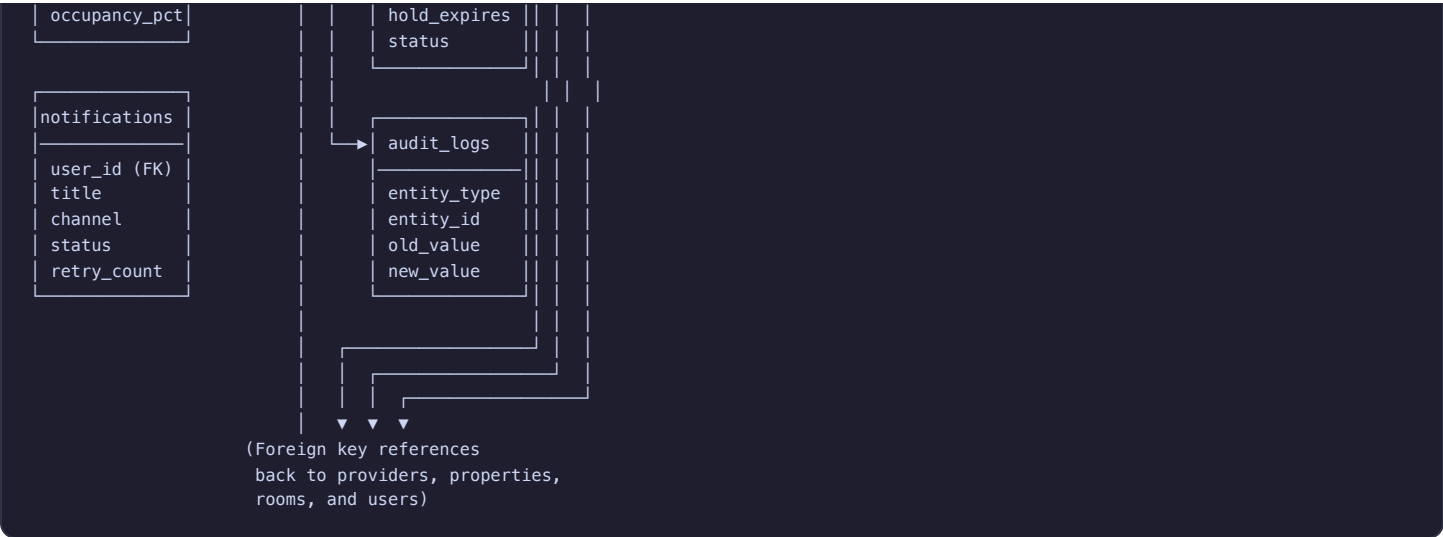
```
CREATE TYPE property_type AS ENUM ('pg', 'hostel', 'flat', 'room');
CREATE TYPE refund_policy AS ENUM ('full', 'partial', 'none');
CREATE TYPE charge_frequency AS ENUM ('monthly', 'quarterly', 'one_time');

CREATE TYPE booking_status AS ENUM (
    'payment_pending',
    'payment_failed',
    'payment_success',
    'inventory_reserved',
    'confirmed',
    'check_in_pending',
    'checked_in',
    'checked_out',
    'cancelled',
    'refunded',
    'expired',
    'disputed'
);

CREATE TYPE reservation_status AS ENUM ('held', 'confirmed', 'expired', 'released', 'cancelled');
CREATE TYPE payment_status AS ENUM ('pending', 'authorized', 'captured', 'failed', 'refunded', 'partial_refund');
CREATE TYPE notification_type AS ENUM (
    'payment_success', 'booking_created', 'booking_cancelled',
    'refund_initiated', 'refund_completed', 'move_in_reminder',
    'move_out_reminder', 'inventory_low', 'property_full',
    'offline_conflict', 'new_booking', 'provider_action_required'
);
CREATE TYPE notification_channel AS ENUM ('push', 'sms', 'whatsapp', 'email', 'in_app');
CREATE TYPE inventory_event AS ENUM (
    'reservation_hold', 'reservation_release', 'check_in', 'check_out',
    'offline_occupy', 'offline_vacate', 'block_maintenance',
    'unblock_maintenance', 'manual_adjustment', 'reconciliation'
);
```

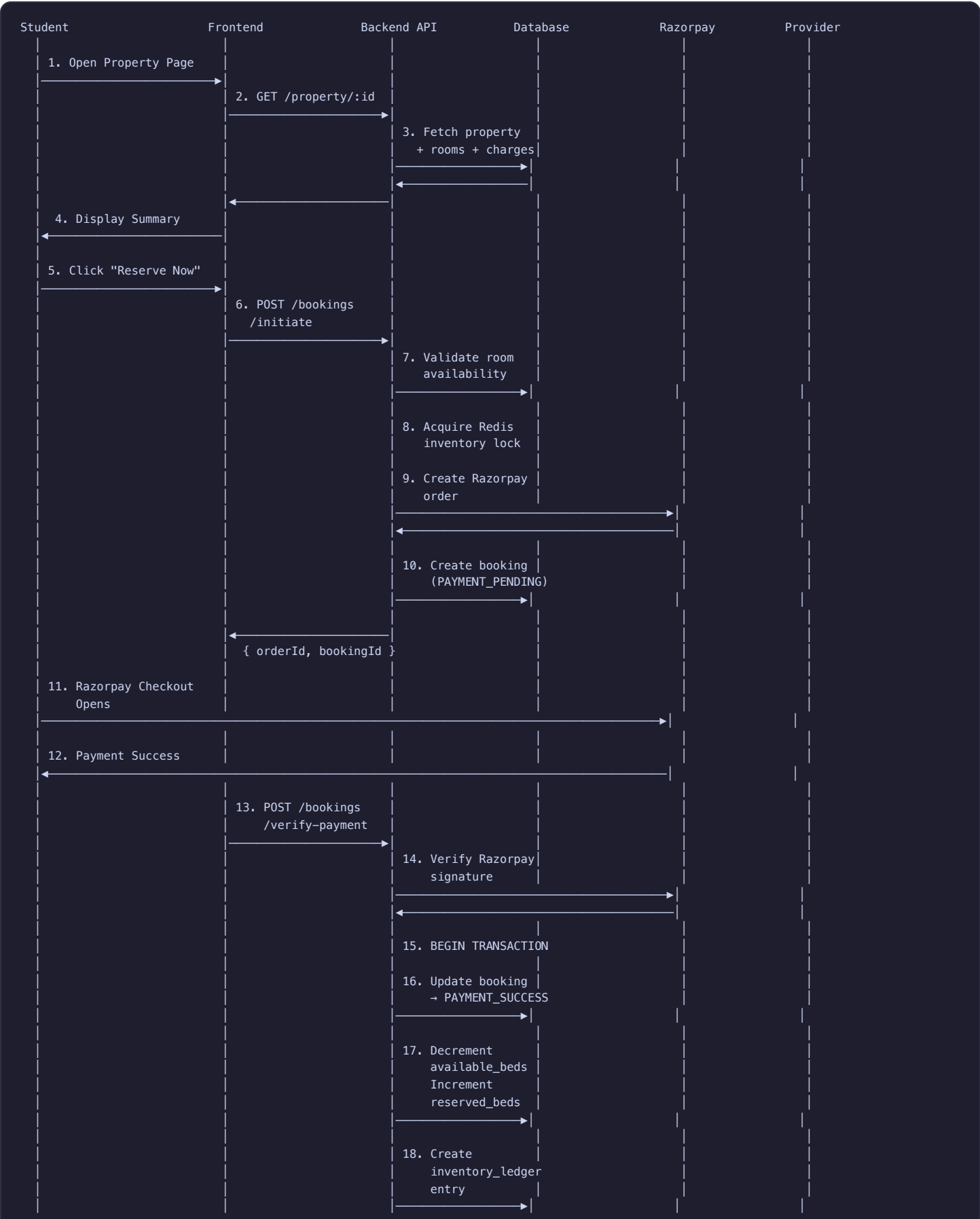
6. Entity Relationship Diagram (ERD)





7. Booking Flow — End to End

7.1 Complete Booking Sequence





7.2 Booking ID Generation

```
Format: STV-{YEAR}-{SEQUENCE}
Example: STV-2026-000001

Algorithm:
1. Use PostgreSQL sequence: CREATE SEQUENCE booking_seq START 1;
2. Pad to 6 digits: LPAD(nextval('booking_seq')::TEXT, 6, '0')
3. Prepend prefix: 'STV-' || EXTRACT(YEAR FROM NOW()) || '-' || padded_sequence
4. Store in booking_id column (indexed, unique)
```

Pseudo Code:

```
async function generateBookingId(prisma: PrismaClient): Promise<string> {
  const result = await prisma.$queryRaw`
    SELECT 'STV-' || EXTRACT(YEAR FROM NOW())::INT || '-' ||
      LPAD(nextval('booking_seq')::TEXT, 6, '0') AS booking_id
  `;
  return result[0].booking_id;
}
```

7.3 Receipt Generation

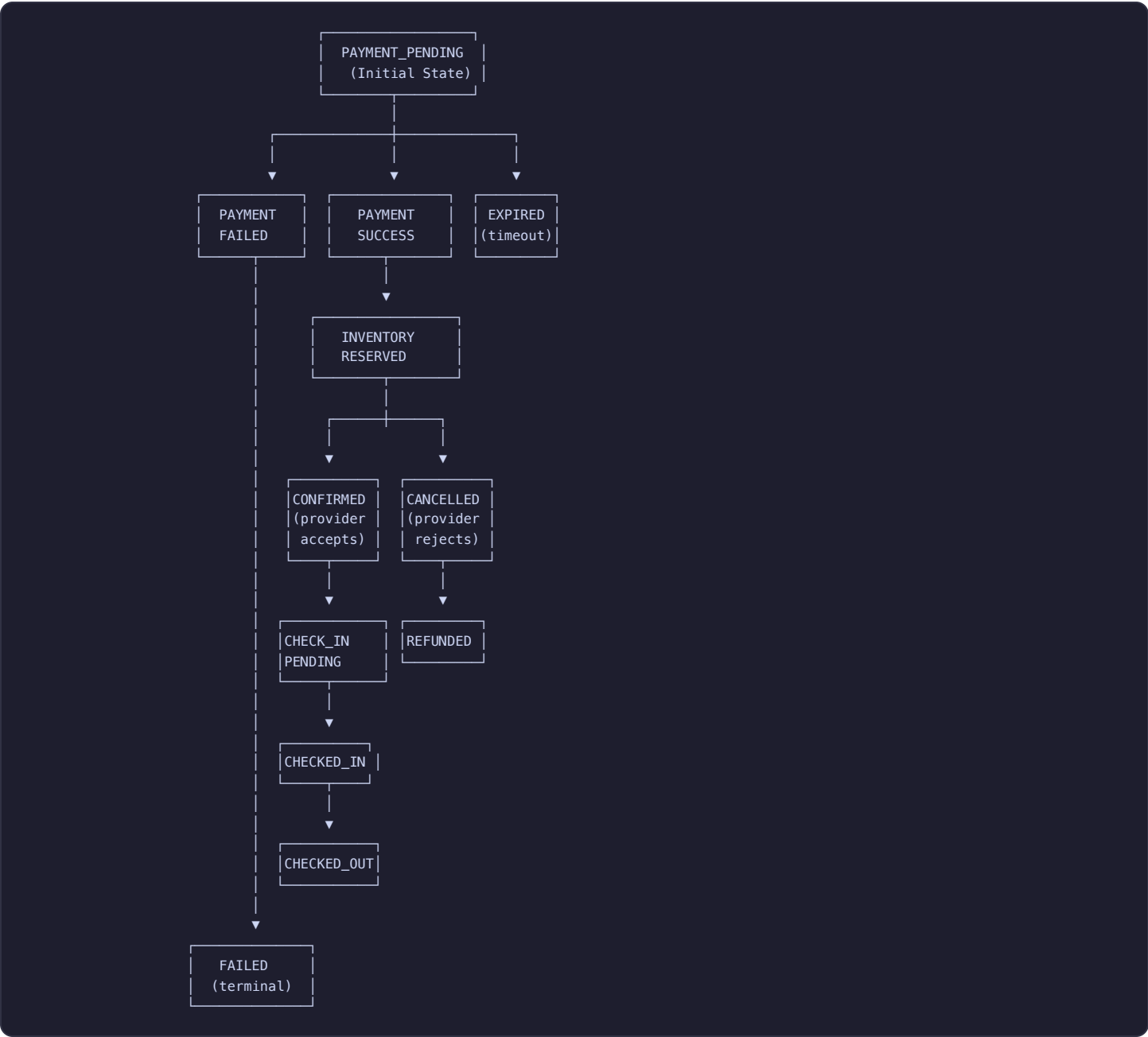
The receipt contains:

FIELD	SOURCE
Booking ID	Generated (STV-2026-XXXXXX)

FIELD	SOURCE
Invoice Number	Generated (STV-INV-2026-XXXXXX)
Property Name	properties.name
Property Address	properties.address
Room Type	property_rooms.room_type
Student Name	student_profiles.full_name
Student Phone	users.phone_number
Reservation Fee	bookings.reservation_amount
Platform Fee	bookings.platform_fee
Total Paid	bookings.total_paid
Pending Amount	bookings.pending_amount
Move-in Date	bookings.move_in_date
Provider Contact	providers.phone_number
QR Code	Encoded URL: https://stayveo.com/verify/{booking_id}
Payment Method	booking_payments.payment_method
Transaction ID	booking_payments.razorpay_payment_id

8. Booking State Machine

8.1 State Diagram



8.2 State Transition Table

FROM STATE	TO STATE	TRIGGER EVENT	ACTOR	SIDE EFFECTS
—	PAYMENT_PENDING	Student clicks "Reserve Now"	System	Create booking record, Razorpay order
PAYMENT_PENDING	PAYMENT_SUCCESS	Razorpay payment confirmed	System	Update payment record
PAYMENT_PENDING	PAYMENT_FAILED	Payment fails/times out	System	Release any pre-locks

FROM STATE	TO STATE	TRIGGER EVENT	ACTOR	SIDE EFFECTS
PAYMENT_PENDING	EXPIRED	30-minute timeout	System (cron)	Auto-cancel booking
PAYMENT_SUCCESS	INVENTORY_RESERVED	Inventory successfully held	System	Decrement <code>available_beds</code> , increment <code>reserved_beds</code> , create ledger entry
INVENTORY_RESERVED	CONFIRMED	Provider accepts booking	Provider	Send confirmation to student
INVENTORY_RESERVED	CANCELLED	Provider rejects / Student cancels	Provider/Student	Release inventory hold, initiate refund
CONFIRMED	CHECK_IN_PENDING	3 days before move-in date	System (cron)	Send move-in reminder
CHECK_IN_PENDING	CHECKED_IN	Student checks in at property	Provider	Decrement <code>reserved_beds</code> , increment <code>occupied_beds</code>
CHECKED_IN	CHECKED_OUT	Student vacates	Provider	Decrement <code>occupied_beds</code> , increment <code>available_beds</code>
CANCELLED	REFUNDED	Refund processed by Razorpay	System	Update refund status
Any active state	CANCELLED	Admin cancellation	Admin	Full cleanup + refund

8.3 State Machine Implementation

```
// Domain: BookingStateMachine

type BookingState =
| 'PAYMENT_PENDING'
| 'PAYMENT_SUCCESS'
| 'PAYMENT_FAILED'
| 'INVENTORY_RESERVED'
| 'CONFIRMED'
| 'CHECK_IN_PENDING'
| 'CHECKED_IN'
| 'CHECKED_OUT'
| 'CANCELLED'
| 'REFUNDED'
| 'EXPIRED'
| 'FAILED';

type BookingEvent =
| 'PAYMENT_RECEIVED'
| 'PAYMENT_TIMEOUT'
| 'PAYMENT_ERROR'
| 'INVENTORY_HELD'
| 'INVENTORY_FAILED'
| 'PROVIDER_CONFIRMED'
| 'PROVIDER_REJECTED'
| 'STUDENT_CANCELLED'
| 'CHECK_IN_WINDOW'
| 'STUDENT_CHECKED_IN'
| 'STUDENT_CHECKED_OUT'
| 'REFUND_COMPLETED'
| 'ADMIN_CANCEL'
| 'HOLD_EXPIRED';

const TRANSITIONS: Record<BookingState, Partial<Record<BookingEvent, BookingState>>> = {
  PAYMENT_PENDING: {
    PAYMENT_RECEIVED: 'PAYMENT_SUCCESS',
    PAYMENT_TIMEOUT: 'EXPIRED',
    PAYMENT_ERROR: 'PAYMENT_FAILED',
  },
  PAYMENT_SUCCESS: {
    INVENTORY_HELD: 'INVENTORY_RESERVED',
    INVENTORY_FAILED: 'FAILED',
  },
  INVENTORY_RESERVED: {
    PROVIDER_CONFIRMED: 'CONFIRMED',
    PROVIDER_REJECTED: 'CANCELLED',
    STUDENT_CANCELLED: 'CANCELLED',
    HOLD_EXPIRED: 'CANCELLED',
    ADMIN_CANCEL: 'CANCELLED',
  },
  CONFIRMED: {
    CHECK_IN_WINDOW: 'CHECK_IN_PENDING',
    STUDENT_CANCELLED: 'CANCELLED',
    ADMIN_CANCEL: 'CANCELLED',
  },
  CHECK_IN_PENDING: {
    STUDENT_CHECKED_IN: 'CHECKED_IN',
    STUDENT_CANCELLED: 'CANCELLED',
    ADMIN_CANCEL: 'CANCELLED',
  },
  CHECKED_IN: {
    STUDENT_CHECKED_OUT: 'CHECKED_OUT',
  },
  CANCELLED: {
    REFUND_COMPLETED: 'REFUNDED',
  },
  // Terminal states – no transitions
  PAYMENT_FAILED: {},
  CHECKED_OUT: {},
  REFUNDED: {},
  EXPIRED: {},
  FAILED: {},
};
```

```
class BookingStateMachine {
  transition(currentState: BookingState, event: BookingEvent): BookingState {
    const nextState = TRANSITIONS[currentState]?.[event];
    if (!nextState) {
      throw new InvalidStateTransitionError(
        `Cannot transition from ${currentState} via ${event}`
      );
    }
    return nextState;
  }

  canTransition(currentState: BookingState, event: BookingEvent): boolean {
    return !!TRANSITIONS[currentState]?.[event];
  }
}
```

9. Inventory Management System

9.1 Architecture Overview



9.2 When Should Inventory Decrease?

APPROACH	WHEN	PROS	CONS	VERDICT
Immediately after payment	Payment webhook received	Fastest reservation, no double-booking risk	Provider hasn't confirmed yet; could lead to unconfirmed reservations holding beds	 RECOMMENDED
After provider confirmation	Provider clicks "Accept"	Provider has full control	Gap between payment and confirmation = possible overbooking; student waits anxiously	 Risky
After move-in	Student physically arrives	Most "real" — reflects actual occupancy	Beds appear available for days/weeks after they're reserved = massive overbooking risk	 Dangerous

Decision: Immediately after payment (with reservation hold)

The inventory decrements from `available_beds` and increments `reserved_beds` the moment payment is verified. This provides:

- Zero overbooking risk for online bookings
- Clear separation: `reserved ≠ occupied` — the provider knows exactly which beds are reserved vs. occupied
- If the provider rejects or the student cancels, the hold is released and beds go back to `available`

9.3 Reservation Hold Logic

```
class ReservationHoldService {
  private readonly HOLD_TTL_HOURS = 72; // 3 days default
  private readonly LOCK_TTL_MS = 30_000; // 30 seconds
  private readonly MAX_RETRIES = 3;

  async acquireHold(
    roomId: string,
    bookingId: string,
    beds: number = 1
  ): Promise<ReservationHold> {
    // Step 1: Acquire distributed lock (Redis)
    const lockKey = `inventory:lock:${roomId}`;
    const lock = await this.redis.set(lockKey, bookingId, 'PX', this.LOCK_TTL_MS, 'NX');

    if (!lock) {
      throw new InventoryLockError('Room inventory is being modified by another operation');
    }

    try {
      // Step 2: Begin database transaction
      return await this.prisma.$transaction(async (tx) => {
        // Step 3: Read current inventory WITH row-level lock
        const room = await tx.propertyRoom.findUnique({
          where: { id: roomId },
          // SELECT ... FOR UPDATE
        });

        if (!room || room.available_beds < beds) {
          throw new InsufficientInventoryError(
            `Room ${roomId} has ${room?.available_beds ?? 0} available beds, need ${beds}`
          );
        }

        // Step 4: Decrement available, increment reserved
        await tx.propertyRoom.update({
          where: { id: roomId },
          data: {
            available_beds: { decrement: beds },
            reserved_beds: { increment: beds },
          },
        });

        // Step 5: Create reservation ledger entry
        const hold = await tx.reservationLedger.create({
          data: {
            bookingId,
            roomId,
            bedsHeld: beds,
            holdStartedAt: new Date(),
            holdExpiresAt: new Date(Date.now() + this.HOLD_TTL_HOURS * 3600_000),
            status: 'active',
          },
        });

        // Step 6: Create inventory ledger entry
        await tx.inventoryLedger.create({
          data: {
            roomId,
            propertyId: room.property_id,
            eventType: 'reservation_hold',
            bedsDelta: -beds,
            fieldAffected: 'available',
            availableAfter: room.available_beds - beds,
            reservedAfter: room.reserved_beds + beds,
            occupiedAfter: room.occupied_beds,
            offlineAfter: room.offline_beds,
            blockedAfter: room.blocked_beds,
            bookingId,
            source: 'system',
          },
        });

        return hold;
      });
    }
  }
}
```

```

    }, {
      isolationLevel: 'Serializable',
    });
  } finally {
    // Step 7: Always release Redis lock
    await this.redis.del(lockKey);
  }
}

async releaseHold(bookingId: string, reason: string): Promise<void> {
  await this.prisma.$transaction(async (tx) => {
    const hold = await tx.reservationLedger.findFirst({
      where: { bookingId, status: 'active' },
    });

    if (!hold) return;

    // Release beds: reserved → available
    await tx.propertyRoom.update({
      where: { id: hold.roomId },
      data: {
        reserved_beds: { decrement: hold.bedsHeld },
        available_beds: { increment: hold.bedsHeld },
      },
    });

    // Update reservation ledger
    await tx.reservationLedger.update({
      where: { id: hold.id },
      data: {
        status: 'released',
        resolvedAt: new Date(),
        resolutionReason: reason,
      },
    });

    // Inventory ledger
    await tx.inventoryLedger.create({
      data: {
        roomId: hold.roomId,
        eventType: 'reservation_release',
        bedsDelta: hold.bedsHeld,
        fieldAffected: 'available',
        bookingId,
        source: 'system',
        reason,
      },
    });
  });
}
}

```

9.4 Race Condition Handling

Problem: Two students try to book the last available bed simultaneously.

Solution: Three-layer protection:

```

Layer 1: Redis Distributed Lock
├─ SET inventory:lock:{room_id} {booking_id} PX 30000 NX
├─ If lock fails → return 409 Conflict
└─ Always release in finally block

Layer 2: PostgreSQL Row-Level Lock
├─ SELECT ... FOR UPDATE on property_rooms
├─ Only one transaction can hold the row lock
└─ Second transaction waits (with timeout)

Layer 3: CHECK Constraint
├─ available_beds >= 0
├─ Database rejects negative inventory
└─ Final safety net

```

```
// Pseudo-code: Concurrent booking protection
async function bookBed(roomId: string, bookingId: string) {
  // LAYER 1: Redis lock
  const lock = await redis.set(`inv:lock:${roomId}`, bookingId, 'PX', 30000, 'NX');
  if (!lock) throw new ConflictError('Another booking is in progress');

  try {
    // LAYER 2: PostgreSQL serializable transaction
    await prisma.$transaction(async (tx) => {
      // FOR UPDATE row lock
      const room = await tx.$queryRaw`
        SELECT * FROM property_rooms WHERE id = ${roomId} FOR UPDATE
      `;

      if (room.available_beds < 1) {
        throw new InsufficientInventoryError('No beds available');
      }

      // LAYER 3: CHECK constraint protects against negative
      await tx.$queryRaw`
        UPDATE property_rooms
        SET available_beds = available_beds - 1,
            reserved_beds = reserved_beds + 1,
            updated_at = NOW()
        WHERE id = ${roomId}
        AND available_beds > 0 -- additional safety
      `;
    }, { isolationLevel: 'Serializable' });
  } finally {
    await redis.del(`inv:lock:${roomId}`);
  }
}
```

9.5 Inventory Events (Event-Driven)

```
// Events emitted by Inventory Service
type InventoryEvent =
| { type: 'RESERVATION_HELD';    roomId: string; bookingId: string; beds: number }
| { type: 'RESERVATION_RELEASED'; roomId: string; bookingId: string; beds: number; reason: string }
| { type: 'CHECK_IN';          roomId: string; bookingId: string; beds: number }
| { type: 'CHECK_OUT';         roomId: string; bookingId: string; beds: number }
| { type: 'OFFLINE_OCCUPY';     roomId: string; beds: number; providerId: string }
| { type: 'OFFLINE_VACATE';     roomId: string; beds: number; providerId: string }
| { type: 'MAINTENANCE_BLOCK';  roomId: string; beds: number }
| { type: 'MAINTENANCE_UNBLOCK'; roomId: string; beds: number }
| { type: 'INVENTORY_LOW';      roomId: string; availableBeds: number; threshold: number }
| { type: 'PROPERTY_FULL';      propertyId: string }
| { type: 'RECONCILIATION_DRIFT'; roomId: string; expected: InventorySnapshot; actual: InventorySnapshot };

// Event bus (in-process for monolith, Kafka/Redis Streams for microservices)
class InventoryEventBus {
  private handlers: Map<string, Function[]> = new Map();

  on(eventType: string, handler: Function) {
    const handlers = this.handlers.get(eventType) || [];
    handlers.push(handler);
    this.handlers.set(eventType, handlers);
  }

  emit(event: InventoryEvent) {
    const handlers = this.handlers.get(event.type) || [];
    for (const handler of handlers) {
      // Fire and forget – don't block inventory operations
      handler(event).catch(console.error);
    }
  }
}
```

9.6 Overbooking Prevention

OVERBOOKING PREVENTION

1. Application Layer:
if (room.available_beds < requested_beds)
→ throw InsufficientInventoryError
2. Database Layer:
CHECK (available_beds >= 0)
→ PostgreSQL rejects the update
3. Invariant:
CHECK (available + reserved + occupied
+ offline + blocked = total_beds)
→ Drift is impossible at the DB level
4. Concurrency:
Redis Lock → FOR UPDATE → Serializable Txn
→ Only one writer at a time per room
5. Monitoring:
Daily reconciliation cron detects any drift
→ Auto-alerts admin if inventory is inconsistent

9.7 Inventory Rollback Strategy

If a booking fails after inventory was decremented:

```
async function rollbackInventory(bookingId: string, roomId: string) {
  await prisma.$transaction(async (tx) => {
    // 1. Reverse the bed allocation
    await tx.propertyRoom.update({
      where: { id: roomId },
      data: {
        available_beds: { increment: 1 },
        reserved_beds: { decrement: 1 },
      },
    });

    // 2. Log the rollback in inventory ledger
    await tx.inventoryLedger.create({
      data: {
        roomId,
        eventType: 'reservation_release',
        bedsDelta: 1,
        fieldAffected: 'available',
        bookingId,
        source: 'system',
        reason: 'Booking creation failed - inventory rollback',
      },
    });

    // 3. Update reservation ledger
    await tx.reservationLedger.updateMany({
      where: { bookingId, status: 'active' },
      data: { status: 'released', resolvedAt: new Date() },
    });
  });
}
```

10. Offline Inventory Management

10.1 The Problem

PG owners frequently take walk-in customers who pay cash and move in immediately. These offline bookings aren't tracked by the platform, leading to:

- **Phantom availability:** Platform shows 5 beds available, but owner has filled 3 offline
- **Overbooking:** Student pays online, shows up, no bed available
- **Data inconsistency:** Inventory numbers don't match reality

10.2 Solution Architecture — Low-Friction Offline Sync

OFFLINE INVENTORY MANAGEMENT

Strategy: Multi-Channel, Low-Friction Reconciliation

CHANNEL 1: Quick Dashboard Counter

Provider Dashboard shows:

Single Room (70 beds)

Online Booked:	40	Offline Occupied:	[15]
Available:	15	Maintenance:	[0]

[-] 15 [+] ← One-tap adjustment

[Save Changes]

CHANNEL 2: WhatsApp Integration

Every evening at 8 PM, system sends WhatsApp message:

"Hi Rajesh! 🏠 Daily Inventory Check
Green Valley PG – Single Room
We show: 15 beds available

Is this correct?
Reply: YES or the actual number"

Owner replies: "12"
System auto-adjusts: 3 beds marked as offline_occupied

CHANNEL 3: Daily Reconciliation Cron

Runs at: 02:00 AM IST daily

1. Take inventory snapshot
2. Compare with previous day's snapshot
3. Check for drift:
 - Did total beds change? (shouldn't unless room added)
 - Does sum of states = total?
 - Are there expired reservation holds?
4. Auto-release expired holds
5. Flag anomalies for admin review
6. Send summary to provider if drift detected

CHANNEL 4: Calendar Sync (Future)

Provider marks move-in/move-out dates on calendar
System auto-adjusts inventory based on calendar events

10.3 Quick Inventory Sync — Implementation

```
// Provider Dashboard: One-click offline adjustment
async function syncOfflineInventory(
  roomId: string,
  providerId: string,
  actualOfflineBeds: number
): Promise<void> {
  await prisma.$transaction(async (tx) => {
    const room = await tx.propertyRoom.findUnique({ where: { id: roomId } });

    const currentOffline = room.offline_beds;
    const delta = actualOfflineBeds - currentOffline;

    if (delta === 0) return; // No change

    // Offline beds increase → available beds decrease (and vice versa)
    await tx.propertyRoom.update({
      where: { id: roomId },
      data: {
        offline_beds: actualOfflineBeds,
        available_beds: room.available_beds - delta,
        updated_at: new Date(),
      },
    });

    await tx.inventoryLedger.create({
      data: {
        roomId,
        propertyId: room.property_id,
        eventType: delta > 0 ? 'offline_occupy' : 'offline_vacate',
        bedsDelta: Math.abs(delta),
        fieldAffected: 'offline',
        performedBy: providerId,
        source: 'provider',
        reason: `Provider adjusted offline occupancy from ${currentOffline} to ${actualOfflineBeds}`,
      },
    });
  });
}
```

10.4 Daily Reconciliation Cron

```
// Runs daily at 2 AM IST
async function dailyInventoryReconciliation(): Promise<void> {
  const allRooms = await prisma.propertyRoom.findMany({
    where: { is_active: true },
  });

  for (const room of allRooms) {
    const sum = room.available_beds + room.reserved_beds
      + room.occupied_beds + room.offline_beds + room.blocked_beds;

    if (sum !== room.total_beds) {
      // DRIFT DETECTED
      const drift = room.total_beds - sum;

      // Log anomaly
      await prisma.inventoryLedger.create({
        data: {
          roomId: room.id,
          propertyId: room.property_id,
          eventType: 'reconciliation',
          bedsDelta: drift,
          fieldAffected: 'available',
          source: 'cron',
          reason: `Reconciliation drift: expected ${room.total_beds}, got ${sum}. Adjusted available_beds by ${drift}`,
        },
      });

      // Auto-correct by adjusting available_beds
      await prisma.propertyRoom.update({
        where: { id: room.id },
        data: { available_beds: room.available_beds + drift },
      });

      // Notify admin
      await notificationService.sendToAdmin({
        type: 'RECONCILIATION_DRIFT',
        roomId: room.id,
        drift,
      });
    }

    // Take daily snapshot
    await prisma.inventorySnapshot.create({
      data: {
        roomId: room.id,
        propertyId: room.property_id,
        snapshotDate: new Date(),
        totalBeds: room.total_beds,
        availableBeds: room.available_beds,
        reservedBeds: room.reserved_beds,
        occupiedBeds: room.occupied_beds,
        offlineBeds: room.offline_beds,
        blockedBeds: room.blocked_beds,
        occupancyRate: ((room.occupied_beds + room.offline_beds + room.reserved_beds) / room.total_beds) * 100,
      },
    });
  }

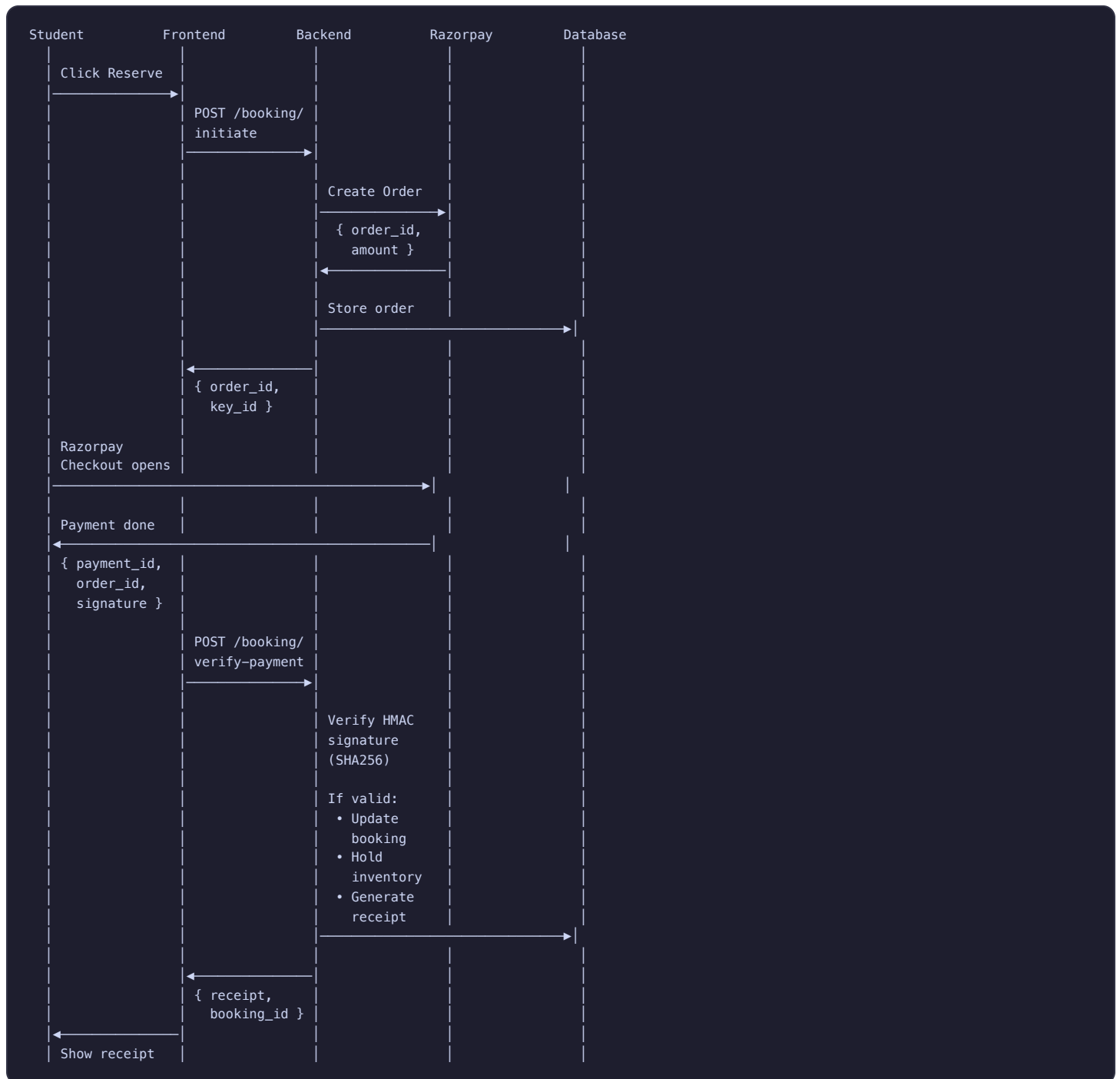
  // Release expired reservation holds
  const expiredHolds = await prisma.reservationLedger.findMany({
    where: {
      status: 'active',
      holdExpiresAt: { lt: new Date() },
    },
  });

  for (const hold of expiredHolds) {
    await reservationHoldService.releaseHold(
      hold.bookingId,
      'Reservation hold expired – auto-released by reconciliation cron'
    );
  }
}
```

```
}  
}
```

11. Payment Architecture

11.1 Payment Flow with Razorpay



11.2 Razorpay Signature Verification

```
import crypto from 'crypto';

function verifyRazorpaySignature(
  orderId: string,
  paymentId: string,
  signature: string,
  secret: string
): boolean {
  const body = orderId + '|' + paymentId;
  const expectedSignature = crypto
    .createHmac('sha256', secret)
    .update(body)
    .digest('hex');

  return crypto.timingSafeEqual(
    Buffer.from(expectedSignature),
    Buffer.from(signature)
  );
}
```

11.3 Webhook Handling

```
// POST /api/v1/payments/webhook
async function handleRazorpayWebhook(request: FastifyRequest, reply: FastifyReply) {
  // 1. Verify webhook signature
  const webhookSignature = request.headers['x-razorpay-signature'] as string;
  const webhookSecret = process.env.RAZORPAY_WEBHOOK_SECRET;

  const isValid = verifyWebhookSignature(
    JSON.stringify(request.body),
    webhookSignature,
    webhookSecret
  );

  if (!isValid) {
    await paymentLogger.log('webhook_failed', { reason: 'Invalid signature' });
    return reply.status(401).send({ error: 'Invalid webhook signature' });
  }

  // 2. Idempotency check – prevent duplicate processing
  const eventId = request.body.event_id;
  const existing = await redis.get(`webhook:${eventId}`);
  if (existing) {
    return reply.status(200).send({ status: 'already_processed' });
  }
  await redis.set(`webhook:${eventId}`, 'processing', 'EX', 86400);

  // 3. Process event
  const event = request.body.event;
  const payload = request.body.payload;

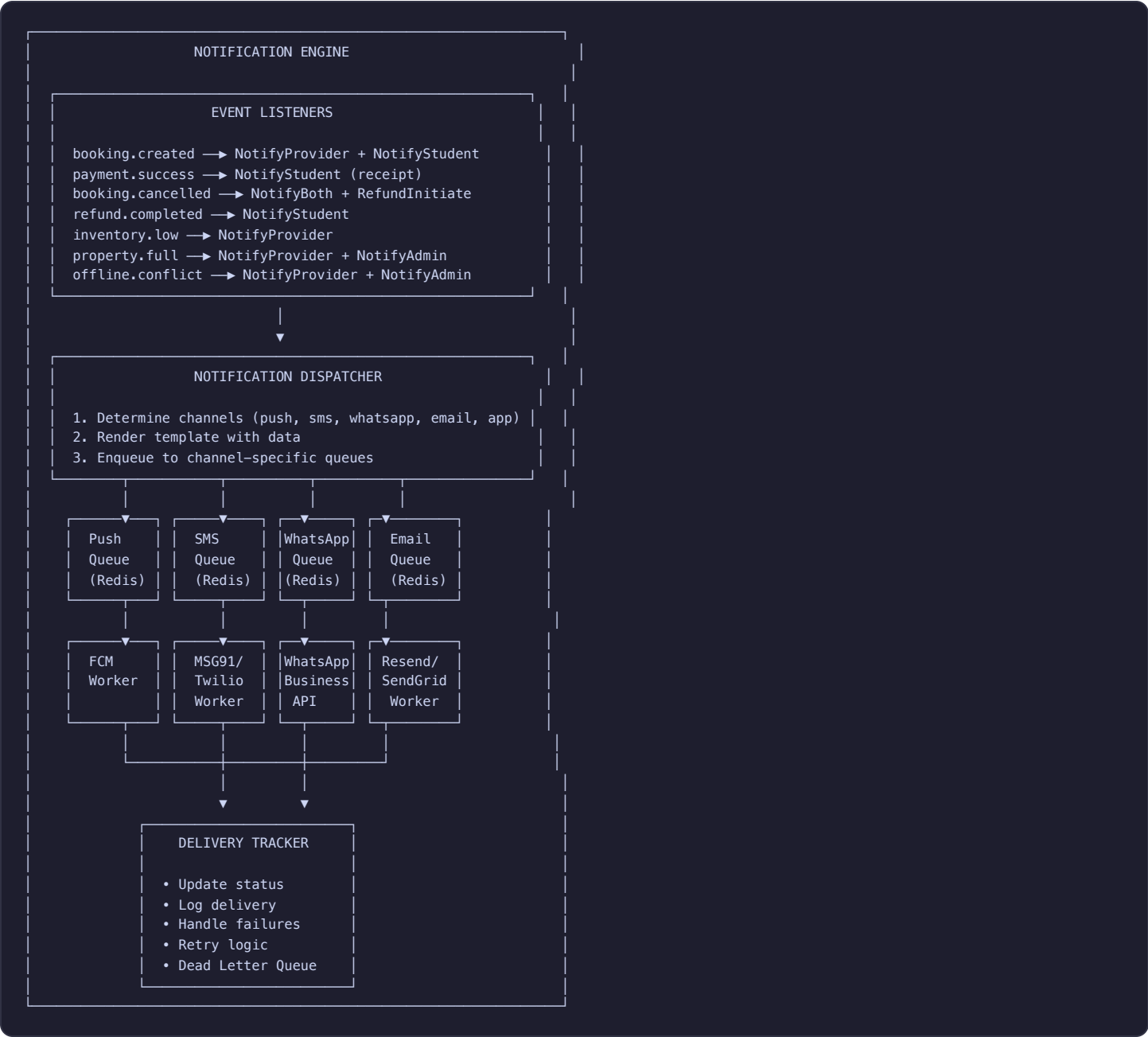
  switch (event) {
    case 'payment.captured':
      await handlePaymentCaptured(payload);
      break;
    case 'payment.failed':
      await handlePaymentFailed(payload);
      break;
    case 'refund.processed':
      await handleRefundProcessed(payload);
      break;
  }

  // 4. Mark as processed
  await redis.set(`webhook:${eventId}`, 'processed', 'EX', 86400);

  return reply.status(200).send({ status: 'ok' });
}
```

12. Notification Engine

12.1 Architecture



12.2 Notification Events Matrix

EVENT	STUDENT	PROVIDER	ADMIN	CHANNELS
Payment Success	✓	✗	✗	Push, SMS, Email, In-App
Booking Created	✓	✓	✗	Push, SMS, WhatsApp, Email, In-App
Booking Cancelled	✓	✓	✗	Push, SMS, Email, In-App
Refund Initiated	✓	✗	✓	Push, Email, In-App

EVENT	STUDENT	PROVIDER	ADMIN	CHANNELS
Refund Completed	✓	✗	✗	Push, SMS, Email, In-App
Move-in Reminder	✓	✓	✗	Push, WhatsApp, In-App
Move-out Reminder	✓	✓	✗	Push, WhatsApp, In-App
Inventory Low	✗	✓	✓	Push, In-App
Property Full	✗	✓	✓	Push, SMS, In-App
Offline Conflict	✗	✓	✓	Push, WhatsApp, In-App

12.3 Retry Strategy

Attempt 1: Immediate
 Attempt 2: After 30 seconds
 Attempt 3: After 2 minutes
 Attempt 4: After 10 minutes (final)

After max attempts → Move to Dead Letter Queue (DLQ)
 DLQ is reviewed by admin manually or processed by a separate worker

```

class NotificationRetryStrategy {
  private readonly RETRY_DELAYS = [0, 30_000, 120_000, 600_000]; // ms

  async processNotification(notification: Notification): Promise<void> {
    const attempt = notification.attempt_count + 1;

    try {
      await this.send(notification);
      await this.markDelivered(notification.id);
    } catch (error) {
      if (attempt >= notification.max_attempts) {
        // Move to DLQ
        await this.moveToDLQ(notification, error);
        return;
      }

      // Schedule retry
      const delay = this.RETRY_DELAYS[attempt] || 600_000;
      await this.scheduleRetry(notification.id, attempt, delay, error.message);
    }
  }

  private async moveToDLQ(notification: Notification, error: Error) {
    await prisma.notification.update({
      where: { id: notification.id },
      data: {
        delivery_status: 'dlq',
        last_error: error.message,
        attempt_count: notification.max_attempts,
      },
    });

    // Alert admin
    await this.alertAdmin('NOTIFICATION_DLQ', {
      notificationId: notification.id,
      type: notification.type,
      error: error.message,
    });
  }
}

```

12.4 Notification Scheduler

```
// Scheduled notifications (move-in reminders, etc.)
class NotificationScheduler {
  // Runs every hour
  async processScheduledNotifications(): Promise<void> {
    const now = new Date();

    // Move-in reminders: 3 days before move-in
    const upcomingMoveIns = await prisma.booking.findMany({
      where: {
        status: { in: ['CONFIRMED', 'CHECK_IN_PENDING'] },
        move_in_date: {
          gte: now,
          lte: new Date(now.getTime() + 3 * 24 * 60 * 60 * 1000),
        },
      },
    });

    for (const booking of upcomingMoveIns) {
      await notificationService.send({
        userId: booking.student_id,
        type: 'move_in_reminder',
        title: 'Move-in Reminder 🏠 ',
        message: `Your move-in to ${booking.property_name} is in ${daysUntil(booking.move_in_date)} days!`,
        channels: ['push', 'whatsapp', 'in_app'],
      });
    }
  }
}
```

13. API Design

13.1 API Conventions

Base URL: <https://api.stayveo.com/api/v1>
Auth: Bearer JWT in Authorization header
Content-Type: application/json
Rate Limit: 100 req/min (student), 200 req/min (provider)

Response Format:

```
{
  "success": boolean,
  "data": T | null,
  "message": string,
  "meta": {
    // for paginated responses
    "page": number,
    "limit": number,
    "total": number,
    "totalPages": number
  }
}
```

Error Format:

```
{
  "success": false,
  "data": null,
  "message": "Human readable error",
  "error": {
    "code": "INSUFFICIENT_INVENTORY",
    "details": { ... }
  }
}
```

13.2 Provider APIs

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	/provider/properties	Provider	Create new property
GET	/provider/properties	Provider	List provider's properties
GET	/provider/properties/:id	Provider	Get property details
PUT	/provider/properties/:id	Provider	Update property
DELETE	/provider/properties/:id	Provider	Delete property
POST	/provider/properties/:id/rooms	Provider	Add room type
PUT	/provider/properties/:id/rooms/:roomId	Provider	Update room
DELETE	/provider/properties/:id/rooms/:roomId	Provider	Delete room
POST	/provider/properties/:id/charges	Provider	Add custom charge
PUT	/provider/charges/:chargeId	Provider	Update charge
DELETE	/provider/charges/:chargeId	Provider	Delete charge
PATCH	/provider/rooms/:roomId/inventory	Provider	Quick inventory sync
GET	/provider/bookings	Provider	List provider's bookings
PATCH	/provider/bookings/:id/accept	Provider	Accept booking
PATCH	/provider/bookings/:id/reject	Provider	Reject booking
PATCH	/provider/bookings/:id/check-in	Provider	Mark student checked in
PATCH	/provider/bookings/:id/check-out	Provider	Mark student checked out
GET	/provider/dashboard/stats	Provider	Dashboard statistics
GET	/provider/inventory/history	Provider	Inventory change history

13.3 Student APIs

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/properties	Public	Search/list properties
GET	/properties/:id	Public	Get property with summary
GET	/properties/:id/booking-summary	Public	Full booking cost breakdown
POST	/bookings/initiate	Student	Start booking + create order
POST	/bookings/verify-payment	Student	Verify payment + book
GET	/bookings/:id	Student	Get booking details
GET	/bookings/:id/receipt	Student	Get booking receipt
PATCH	/bookings/:id/cancel	Student	Cancel booking
GET	/student/bookings	Student	List student's bookings
GET	/student/dashboard	Student	Student dashboard
POST	/saved	Student	Save/unsave listing
GET	/saved	Student	List saved listings

13.4 Payment APIs

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	/payments/create-order	Student	Create Razorpay order
POST	/payments/verify	Student	Verify Razorpay signature
POST	/payments/webhook	Razorpay	Razorpay webhook callback
GET	/payments/:id	Auth	Get payment details
POST	/payments/:id/refund	Admin	Initiate refund
GET	/payments/history	Auth	Payment history

13.5 Admin APIs

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/admin/dashboard	Admin	Platform statistics
GET	/admin/bookings	Admin	All bookings
GET	/admin/properties	Admin	All properties
PATCH	/admin/properties/:id/verify	Admin	Verify/reject property
GET	/admin/inventory/anomalies	Admin	Inventory drift alerts
PATCH	/admin/inventory/:roomId/adjust	Admin	Manual inventory adjust
GET	/admin/payments/reconciliation	Admin	Payment reconciliation
POST	/admin/refunds	Admin	Process refund
GET	/admin/notifications/dlq	Admin	Failed notifications
POST	/admin/notifications/dlq/:id/retry	Admin	Retry failed notification
GET	/admin/audit-logs	Admin	System audit logs

13.6 Inventory APIs

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/inventory/rooms/:roomId	Provider	Current inventory state
PATCH	/inventory/rooms/:roomId/offline	Provider	Update offline bed count
PATCH	/inventory/rooms/:roomId/block	Provider	Block beds for maintenance
PATCH	/inventory/rooms/:roomId/unblock	Provider	Unblock maintenance beds
GET	/inventory/rooms/:roomId/history	Provider	Inventory change log
GET	/inventory/rooms/:roomId/snapshots	Provider	Daily snapshots
GET	/inventory/property/:propertyId/summary	Provider	Property-wide inventory

13.7 API Example — Booking Initiate

Request:

```
POST /api/v1/bookings/initiate
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

{
  "property_id": "a1b2c3d4-...",
  "room_id": "e5f6g7h8-...",
  "move_in_date": "2026-08-01",
  "student_name": "Rahul Sharma",
  "student_phone": "+919876543210"
}
```

Response (201 Created):

```
{
  "success": true,
  "data": {
    "booking_id": "STV-2026-000042",
    "internal_id": "uuid-...",
    "razorpay_order_id": "order_LkjHgFdSa12345",
    "razorpay_key_id": "rzp_live_xxxxx",
    "amount": 249900,
    "currency": "INR",
    "breakdown": {
      "reservation_fee": 200000,
      "platform_fee": 49900
    },
    "property": {
      "name": "Green Valley PG",
      "address": "123, Koramangala, Bangalore"
    },
    "room": {
      "type": "Double Sharing",
      "rent": 800000
    },
    "expires_at": "2026-07-20T13:08:47Z"
  },
  "message": "Booking initiated. Complete payment within 30 minutes."
}
```

Error Response (409 Conflict):

```
{
  "success": false,
  "data": null,
  "message": "No beds available for this room type",
  "error": {
    "code": "INSUFFICIENT_INVENTORY",
    "details": {
      "room_id": "e5f6g7h8-...",
      "available_beds": 0,
      "requested_beds": 1
    }
  }
}
```

13.8 API Example — Verify Payment

Request:

```
POST /api/v1/bookings/verify-payment
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

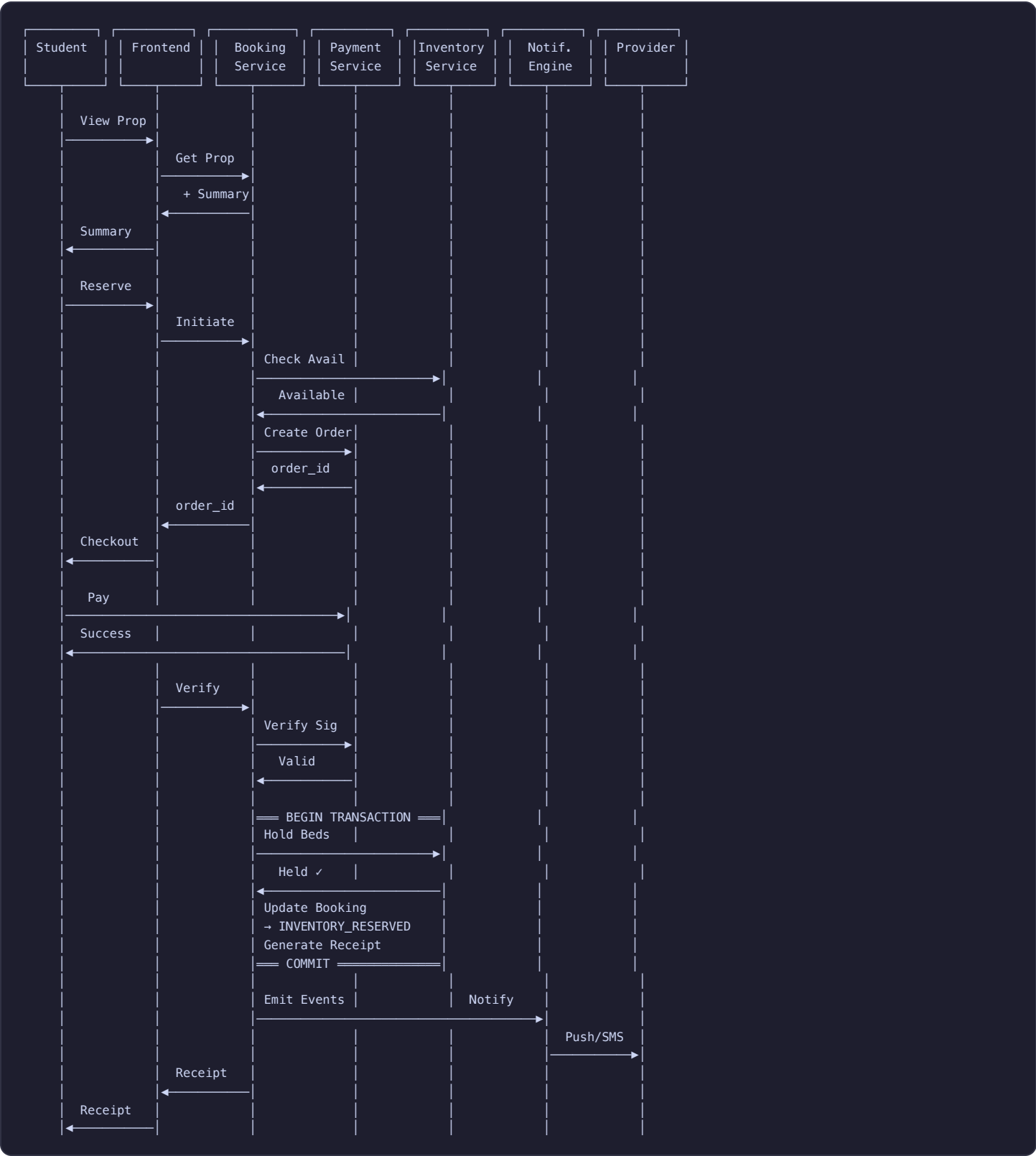
{
  "booking_id": "STV-2026-000042",
  "razorpay_order_id": "order_LkjHgFdSa12345",
  "razorpay_payment_id": "pay_AbCdEfGhIjKl",
  "razorpay_signature": "d5e6f7g8h9i0j1k2l3m4..."
}
```

Response (200 OK):

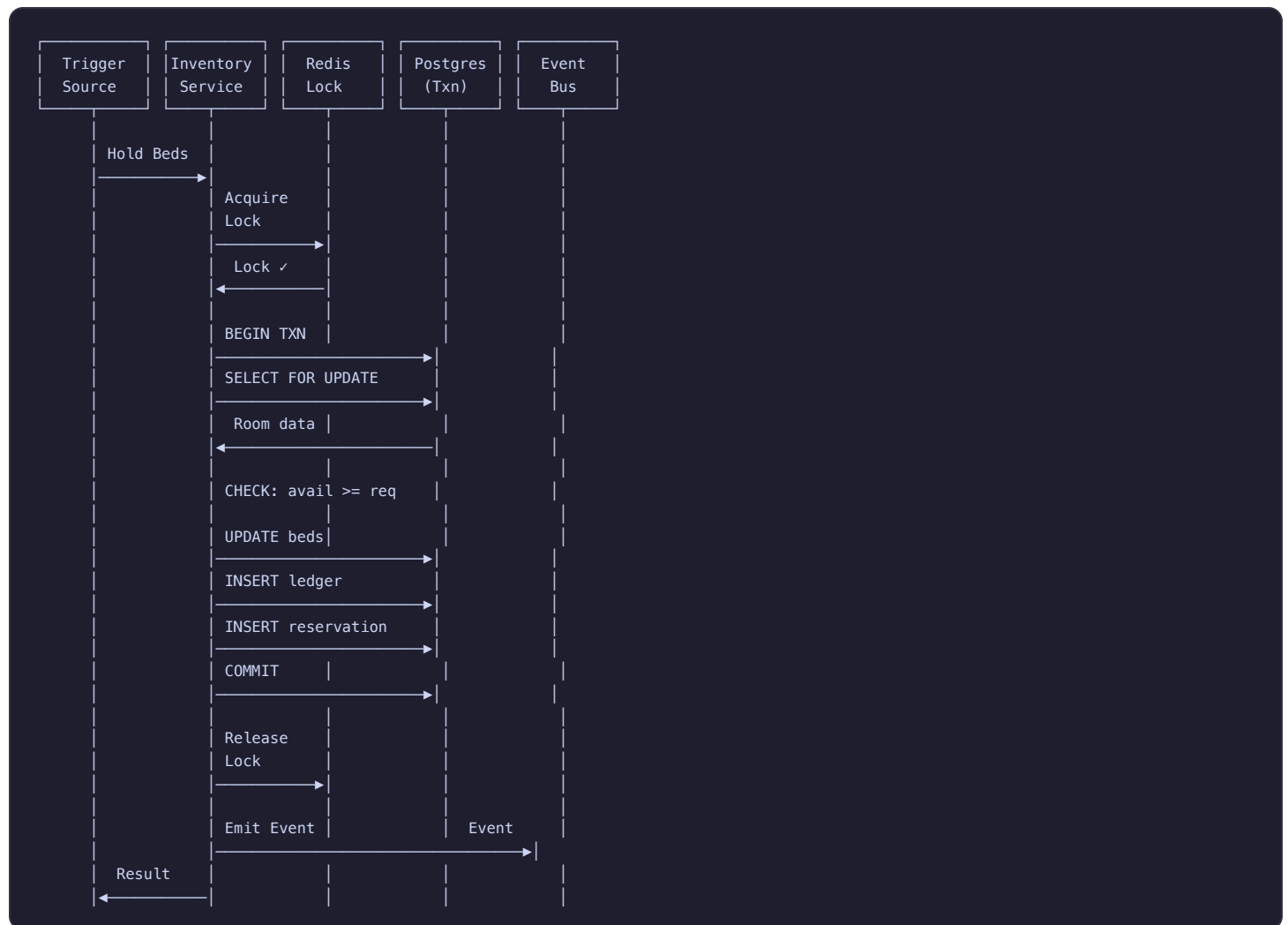
```
{
  "success": true,
  "data": {
    "booking": {
      "booking_id": "STV-2026-000042",
      "status": "INVENTORY_RESERVED",
      "payment_status": "captured",
      "move_in_date": "2026-08-01",
      "created_at": "2026-07-20T12:38:47Z"
    },
    "receipt": {
      "invoice_number": "STV-INV-2026-000042",
      "property_name": "Green Valley PG",
      "property_address": "123, Koramangala, Bangalore",
      "room_type": "Double Sharing",
      "student_name": "Rahul Sharma",
      "reservation_fee": 2000,
      "platform_fee": 499,
      "total_paid": 2499,
      "pending_amount": 16000,
      "monthly_rent": 8000,
      "security_deposit": 8000,
      "move_in_date": "2026-08-01",
      "provider_contact": "+919988776655",
      "qr_code_url": "https://api.stayveo.com/verify/STV-2026-000042",
      "pdf_url": "https://storage.stayveo.com/invoices/STV-INV-2026-000042.pdf"
    },
    "payment": {
      "razorpay_payment_id": "pay_AbCdEfGhIjKl",
      "method": "upi",
      "amount": 2499
    }
  },
  "message": "Payment verified. Your bed is reserved!"
}
```

14. Sequence Diagrams

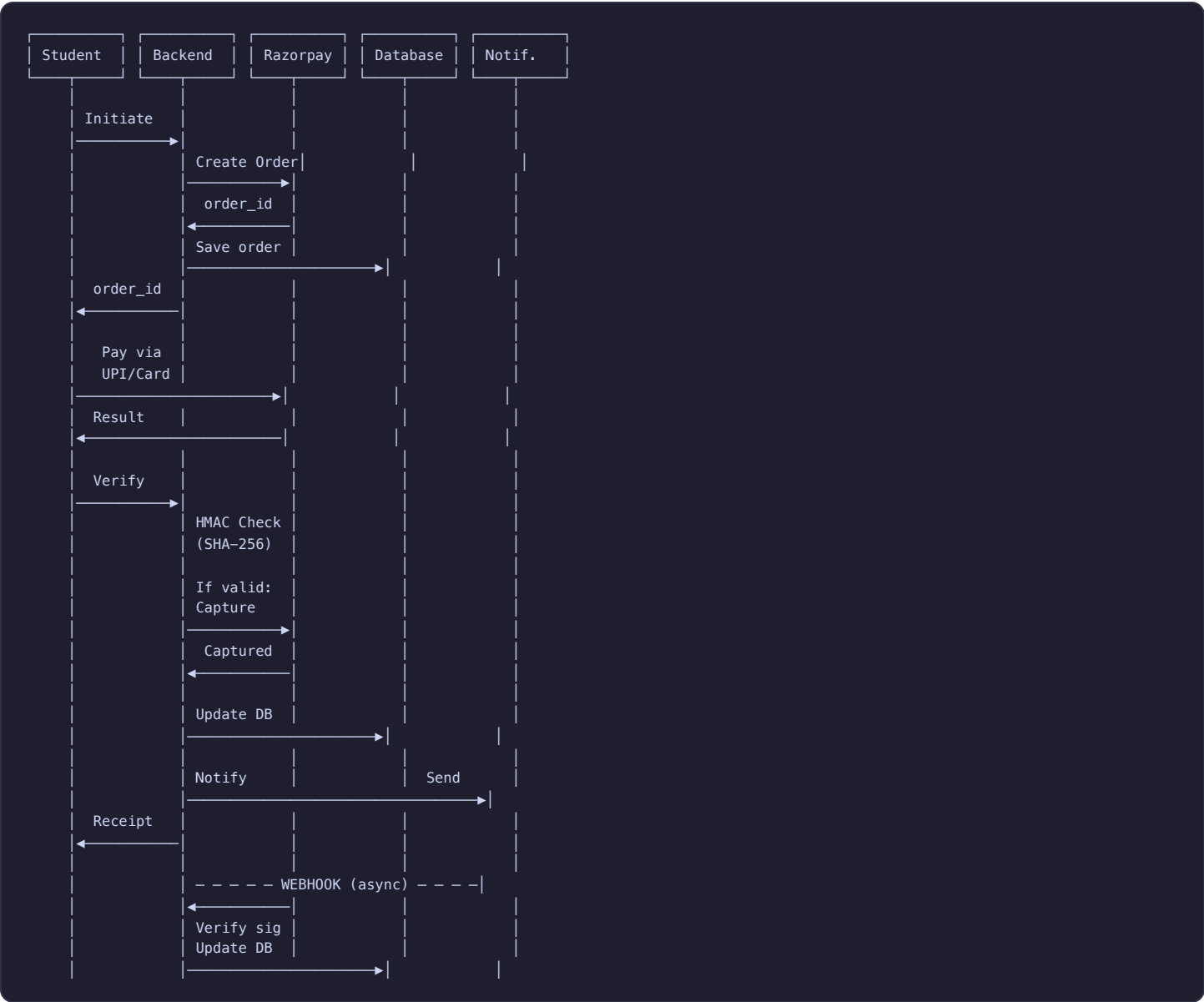
14.1 Student Booking Flow



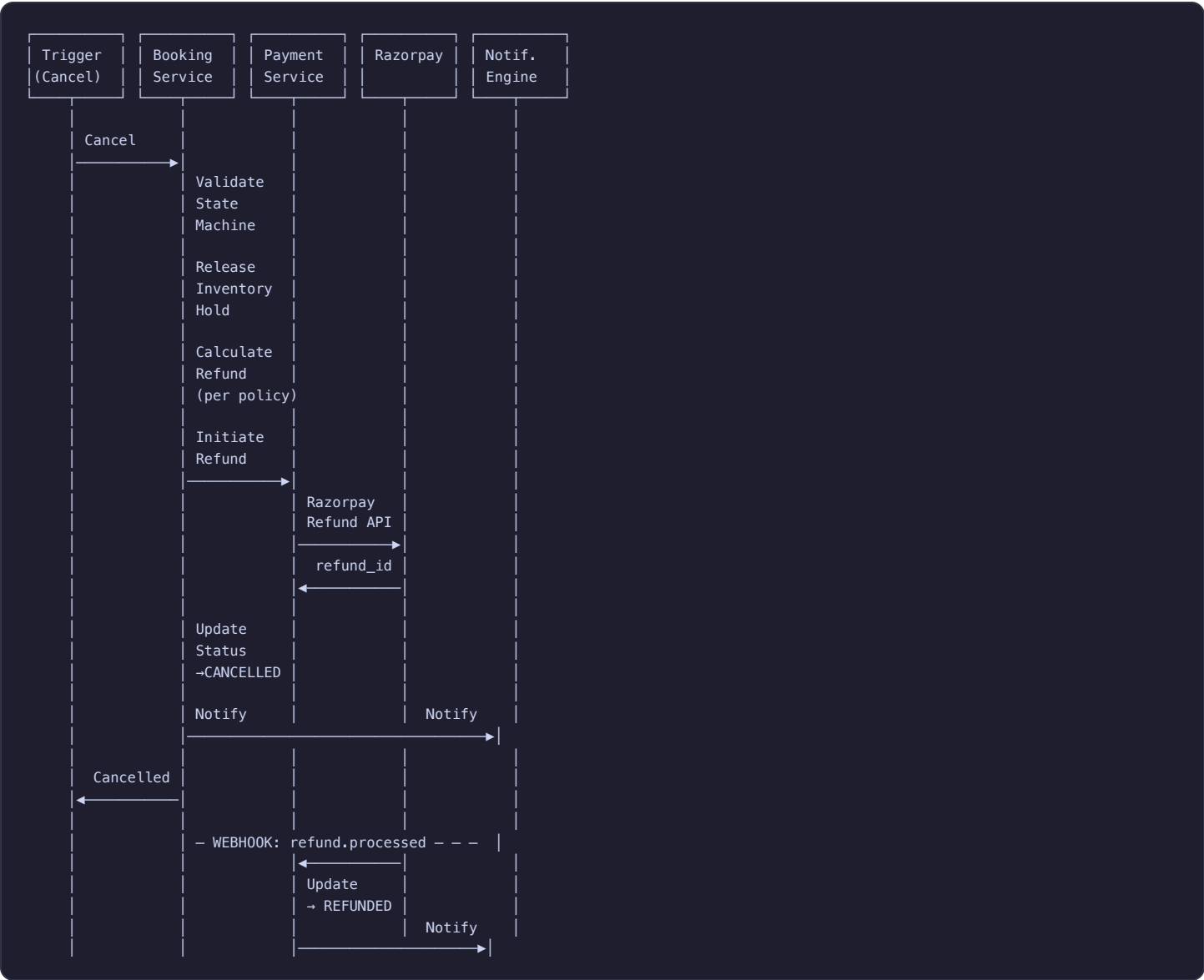
14.2 Inventory Flow



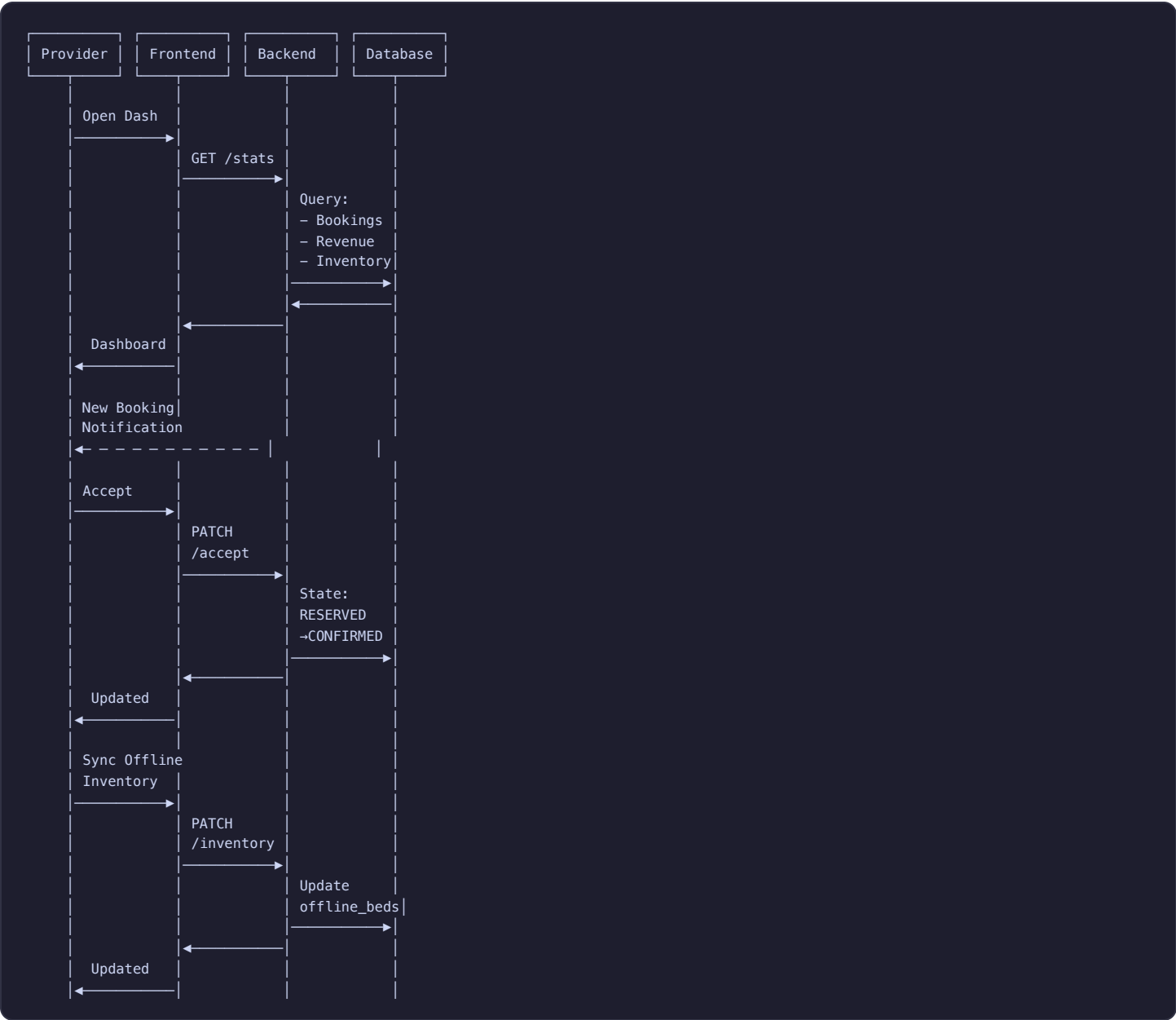
14.3 Payment Flow



14.4 Refund Flow



14.5 Provider Dashboard Flow



15. Class Diagram (UML)

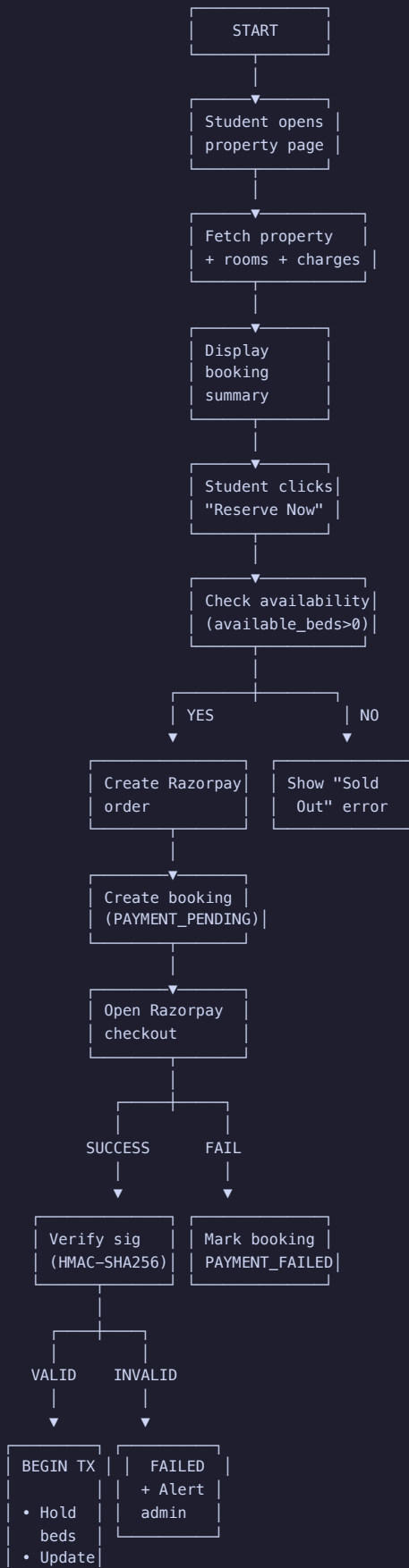


PropertyService
-propertyRepo -chargeRepo
+createProperty() +updateProperty() +addRoom() +addCustomCharge() +getBookingSummary() +search()

«Value Object» BookingSummary
+propertyName: string +roomType: string +rent: Money +securityDeposit: Money +reservationFee: Money +platformFee: Money +charges: Charge[] +totalMonthlyCost: Money +totalPayableNow: Money +pendingAmount: Money

16. Flowcharts

16.1 Booking Flow



```
status
• Log
ledger
COMMIT
```



```
Generate
receipt
+ invoice
+ QR code
```

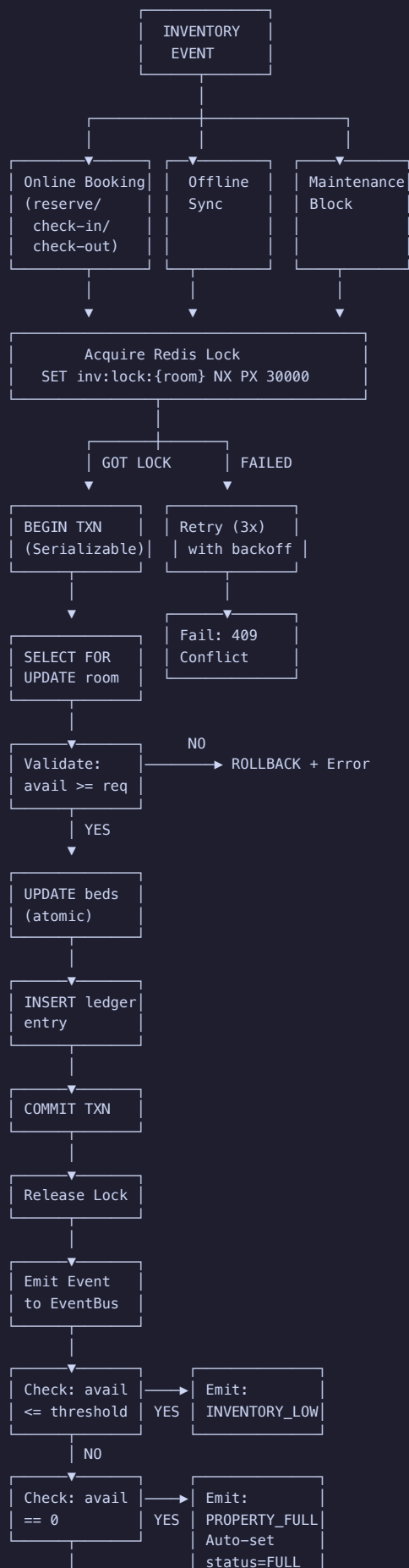


```
Notify
provider
+ student
```



```
END
```

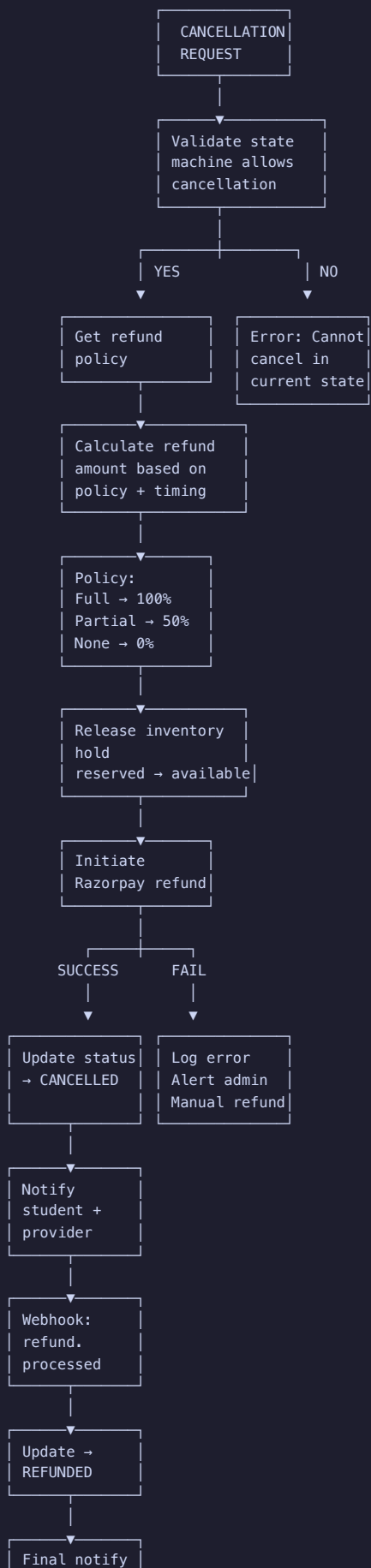
16.2 Inventory Flow



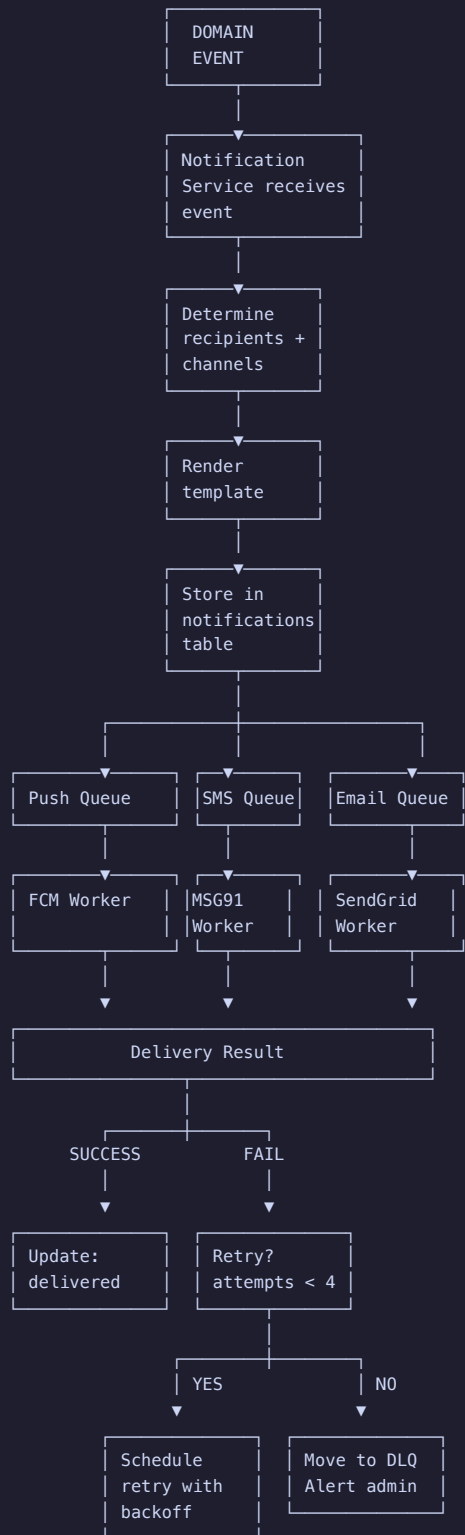


DONE

16.3 Refund Flow



16.4 Notification Flow



17. Security Architecture

17.1 Authentication & Authorization



17.2 Middleware Chain

```
// Request flow through security layers
const securityMiddleware = [
  // 1. Rate Limiter
  rateLimiter({ windowMs: 60000, max: 100, keyGenerator: (req) => req.ip }),

  // 2. Helmet (security headers)
  helmet({
    contentSecurityPolicy: true,
    xContentTypeOptions: true,
    xFrameOptions: 'DENY',
  }),

  // 3. CORS
  cors({ origin: ALLOWED_ORIGINS, credentials: true }),

  // 4. Body parser with size limit
  bodyParser({ limit: '1mb' }),

  // 5. JWT verification (on protected routes)
  jwtVerify({ secret: JWT_SECRET, algorithms: ['HS256'] }),

  // 6. Role authorization
  authorize(['student', 'provider', 'admin']),

  // 7. Request logging (sanitized)
  requestLogger({ redact: ['authorization', 'phone_number'] }),
];
```

18. Scalability & Performance

18.1 Growth Projections

Metric	MVP (6 Months)	Growth (1 Year)	Scale (2+ Years)
Users	10K	100K	1M+
Properties	500	10K	100K+
Monthly Bookings	2K	50K	500K+
Daily API Calls	50K	1M	10M+

18.2 Scaling Architecture

SCALING STRATEGY

PHASE 1 – MVP (Single Server)

- Single VPS (4 CPU, 16GB RAM)
 - Fastify (Node.js)
 - PostgreSQL (Supabase managed)
 - Redis (single instance for cache + locks)
 - Supabase Storage for media

PHASE 2 – Growth (Horizontal App, Read Replicas)

- Load Balancer (Nginx/ALB)
 - └─ App Server 1 (Fastify)
 - └─ App Server 2 (Fastify)
 - └─ App Server 3 (Fastify)
- PostgreSQL:
 - └─ Primary (writes)
 - └─ Read Replica 1 (student queries)
 - └─ Read Replica 2 (analytics)
- Redis Cluster:
 - └─ Cache (property data, search results)
 - └─ Sessions
 - └─ Inventory locks + rate limits
- ElasticSearch:
 - └─ Property search with geo-queries
- CDN (Cloudflare):
 - └─ Static assets + property images

PHASE 3 – Scale (Microservices + Event Streaming)

- API Gateway (Kong / AWS API GW)
 - └─ Booking Service (3 instances)
 - └─ Inventory Service (3 instances)
 - └─ Payment Service (2 instances)
 - └─ Notification Service (2 instances)
 - └─ Search Service (2 instances)
- Message Queue:
 - └─ Kafka / Redis Streams for event-driven communication
- Database:
 - └─ Booking DB (sharded by created_at)
 - └─ Inventory DB (sharded by property_id)
 - └─ Analytics DB (columnar – ClickHouse/TimescaleDB)

18.3 Caching Strategy

REDIS CACHING LAYERS

```
Layer 1: Property Cache (TTL: 5 min)
Key: property:{id}
Value: Full property data with rooms + charges
Invalidation: On property update

Layer 2: Search Results Cache (TTL: 2 min)
Key: search:{hash_of_params}
Value: Search results page
Invalidation: Short TTL (search freshness)

Layer 3: Inventory Cache (TTL: 30 sec)
Key: inventory:{room_id}
Value: { available, reserved, occupied, ... }
Invalidation: On any inventory change

Layer 4: Session Cache (TTL: 15 min)
Key: session:{user_id}
Value: User profile + permissions

Layer 5: Rate Limit Counters (TTL: 1 min)
Key: rate:{ip}:{endpoint}
Value: Request count
```

18.4 Database Optimization

```
-- Partitioning strategy for bookings (at scale)
CREATE TABLE bookings (
  id UUID NOT NULL,
  booking_id VARCHAR(20) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  -- ... other columns
) PARTITION BY RANGE (created_at);

CREATE TABLE bookings_2026_q3 PARTITION OF bookings
  FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');

CREATE TABLE bookings_2026_q4 PARTITION OF bookings
  FOR VALUES FROM ('2026-10-01') TO ('2027-01-01');

-- Materialized view for dashboard stats
CREATE MATERIALIZED VIEW provider_stats AS
SELECT
  provider_id,
  COUNT(*) FILTER (WHERE status = 'confirmed') AS active_bookings,
  COUNT(*) FILTER (WHERE status = 'checked_in') AS occupied,
  SUM(total_paid) FILTER (WHERE payment_status = 'captured') AS total_revenue,
  COUNT(DISTINCT property_id) AS properties_count
FROM bookings
GROUP BY provider_id;

-- Refresh every 5 minutes
-- CREATE EXTENSION pg_cron;
-- SELECT cron.schedule('refresh-stats', '* / 5 * * *',
--   'REFRESH MATERIALIZED VIEW CONCURRENTLY provider_stats');
```

19. Edge Cases & Failure Handling

19.1 Edge Case Matrix

#	EDGE CASE	IMPACT	HANDLING STRATEGY
1	Payment Success but Booking Creation Failed	Money taken, no booking	Webhook acts as backup; if booking not found for captured payment, auto-create booking. Alert admin for manual review. Store raw payment in <code>payment_logs</code> .
2	Booking Created but Inventory Failed	Booking exists but beds not held	Saga pattern: if inventory hold fails within the same transaction, rollback entire booking. If fails after commit, compensating transaction releases the booking and initiates refund.
3	Double Payment	Student charged twice	Idempotency key on Razorpay order. Frontend disables button after first click. Webhook deduplication via <code>event_id</code> in Redis. Auto-refund duplicate.
4	Duplicate Booking	Same student books same room twice	Unique constraint on <code>(student_id, room_id, status)</code> where status is active. Check before creating order.
5	Inventory Race Condition	Two students grab last bed	Redis lock → FOR UPDATE → CHECK constraint. Three-layer protection. Losing request gets 409 Conflict.
6	Offline Booking Conflict	Owner fills bed offline, student books same bed online	Owner's quick sync updates offline count. Daily reconciliation detects drift. If conflict detected, admin notified; student gets priority (has paid).
7	Provider Deletes Property	Active bookings exist	Soft delete only. Property marked <code>is_active = false</code> . Active bookings continue. Cannot delete property with active bookings (hard block).
8	Refund Failure	Razorpay refund API fails	Retry 3 times with exponential backoff. If all retries fail, mark as <code>refund_failed</code> , alert admin, create support ticket. Manual refund via Razorpay dashboard.
9	Webhook Delay	Razorpay webhook arrives late or not at all	Frontend verify-payment endpoint handles immediate confirmation. Webhook is a fallback. Cron job checks for payments where webhook hasn't arrived in 1 hour.
10	Network Failure	Payment made but verify-payment call fails	Razorpay order_id stored in booking. Client retries verify-payment. Webhook handles server-side confirmation. 30-min timeout auto-checks with Razorpay API.
11	Concurrent Booking	Multiple students booking the last 3 beds simultaneously	Serializable isolation level. Redis distributed lock per room. Only one transaction executes at a time for a given room.
12	Reservation Hold Expires	Student pays, bed held, but never moves in	Reservation hold has TTL (72 hours by default). Daily cron releases expired holds. Beds return to <code>available_beds</code> .

19.2 Failure Recovery Pseudo-code

```
// Saga: Booking Creation with Compensation
async function createBookingSaga(input: BookingInput): Promise<Booking> {
  let booking: Booking | null = null;
  let inventoryHeld = false;
  let paymentCaptured = false;

  try {
    // Step 1: Verify payment
    const paymentValid = await paymentService.verify(input.razorpaySignature);
    if (!paymentValid) throw new PaymentVerificationError();
    paymentCaptured = true;

    // Step 2: Create booking
    booking = await bookingRepository.create({
      ...input,
      status: 'PAYMENT_SUCCESS',
    });

    // Step 3: Hold inventory (within same DB transaction)
    await inventoryService.holdBeds(input.roomId, booking.id);
    inventoryHeld = true;

    // Step 4: Update booking status
    await bookingRepository.updateStatus(booking.id, 'INVENTORY_RESERVED');

    // Step 5: Generate receipt (non-critical, can fail)
    try {
      await receiptService.generate(booking);
    } catch (e) {
      logger.warn('Receipt generation failed, will retry', { bookingId: booking.id });
      await queue.push('receipt-generation', { bookingId: booking.id });
    }

    // Step 6: Send notifications (async, non-blocking)
    eventBus.emit({ type: 'BOOKING_CREATED', booking });

    return booking;
  } catch (error) {
    // COMPENSATION: Undo in reverse order
    logger.error('Booking saga failed', { error, input });

    if (inventoryHeld && booking) {
      await inventoryService.releaseBeds(booking.id, 'Saga compensation');
    }

    if (booking) {
      await bookingRepository.updateStatus(booking.id, 'FAILED');
    }

    if (paymentCaptured) {
      // Queue refund – don't block on it
      await queue.push('refund', {
        razorpayPaymentId: input.razorpayPaymentId,
        amount: input.amount,
        reason: 'Booking creation failed',
      });
    }

    throw error;
  }
}
```

20. Tech Stack

20.1 Complete Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite 8	SPA with fast HMR
Routing	React Router v7	Client-side routing
Icons	Lucide React	Icon library
Maps	Mapbox GL / React Map GL	Property location display
Backend	Node.js + Fastify 5	High-performance HTTP server
Language	TypeScript 5.8	Type safety
ORM	Prisma 6	Type-safe database access
Database	PostgreSQL 15 (Supabase)	Primary data store
Cache	Redis	Caching, locks, rate limiting, queues
Storage	Supabase Storage	Image/file storage
Auth	Supabase Auth + JWT	Authentication
Payments	Razorpay	Payment gateway
Validation	Zod	Request/response validation
Logging	Pino (via Fastify)	Structured logging
Monitoring	Prometheus + Grafana (future)	Metrics + dashboards
CI/CD	GitHub Actions (future)	Automated deployments
Hosting	Railway / Fly.io / AWS (future)	App hosting

20.2 Dependency Rationale

Decision	Rationale
Fastify over Express	3x faster, built-in validation, schema-based serialization, TypeScript-first
Prisma over Knex/TypeORM	Type-safe queries, auto-generated types, migration tooling, Supabase compatibility
Redis over in-memory	Distributed locks required for inventory. Shared state across multiple app instances.
PostgreSQL over MongoDB	Strong consistency for financial data. ACID transactions critical for inventory + payments.
Zod over Joi	TypeScript-native. Infer types from schemas. Smaller bundle.

21. Folder Structure

```
stayveo/
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma          # Database schema
│   │   ├── migrations/           # Migration history
│   │   └── seed.ts                # Seed data
│   ├── src/
│   │   ├── server.ts              # Entry point
│   │   ├── app.ts                 # App factory (route registration)
│   │   ├── common/
│   │   │   ├── response.ts        # Standardized response format
│   │   │   ├── constants.ts       # App-wide constants
│   │   │   └── types.ts           # Shared TypeScript types
│   │   ├── errors/
│   │   │   ├── handler.ts         # Global error handler
│   │   │   ├── AppError.ts        # Base error class
│   │   │   ├── NotFoundError.ts
│   │   │   ├── ConflictError.ts
│   │   │   └── InsufficientInventoryError.ts
│   │   ├── plugins/
│   │   │   ├── prisma.ts          # Prisma client plugin
│   │   │   ├── redis.ts           # Redis client plugin
│   │   │   └── auth.ts            # JWT auth plugin
│   │   ├── middleware/
│   │   │   ├── authenticate.ts    # JWT verification
│   │   │   ├── authorize.ts       # Role-based access
│   │   │   ├── rateLimiter.ts     # Rate limiting
│   │   │   └── requestLogger.ts    # Request logging
│   │   ├── modules/
│   │   │   ├── auth/
│   │   │   │   ├── auth.routes.ts
│   │   │   │   ├── auth.service.ts
│   │   │   │   └── auth.schema.ts # Zod schemas
│   │   │   ├── bookings/
│   │   │   │   ├── booking.routes.ts
│   │   │   │   ├── booking.service.ts
│   │   │   │   ├── booking.repository.ts
│   │   │   │   ├── booking.schema.ts
│   │   │   │   ├── booking.state-machine.ts
│   │   │   │   └── booking.types.ts
│   │   │   ├── inventory/
│   │   │   │   ├── inventory.routes.ts
│   │   │   │   ├── inventory.service.ts
│   │   │   │   ├── inventory.repository.ts
│   │   │   │   ├── inventory.lock-manager.ts
│   │   │   │   ├── inventory.reconciliation.ts
│   │   │   │   └── inventory.events.ts
│   │   │   ├── payments/
│   │   │   │   ├── payment.routes.ts
│   │   │   │   ├── payment.service.ts
│   │   │   │   ├── payment.gateway.ts # Razorpay integration
│   │   │   │   ├── payment.webhook.ts
│   │   │   │   └── payment.schema.ts
│   │   │   └── notifications/
│   │   │       ├── notification.routes.ts
│   │   │       ├── notification.service.ts
│   │   │       ├── notification.dispatcher.ts
│   │   │       ├── notification.scheduler.ts
│   │   │       ├── notification.retry.ts
│   │   │       └── templates/
│   │   │           ├── booking-created.ts
│   │   │           └── payment-success.ts
```

```

└─ move-in-reminder.ts

└─ properties/
  └─ property.routes.ts
  └─ property.service.ts
  └─ property.repository.ts
  └─ property.schema.ts

└─ provider/
  └─ provider.routes.ts
  └─ provider.service.ts
  └─ provider.schema.ts

└─ student/
  └─ student.routes.ts
  └─ student.service.ts
  └─ student.schema.ts

└─ receipts/
  └─ receipt.service.ts
  └─ receipt.generator.ts

└─ admin/
  └─ admin.routes.ts
  └─ admin.service.ts
  └─ admin.schema.ts

└─ events/
  └─ event-bus.ts          # In-process event emitter
  └─ event-types.ts       # Event type definitions
  └─ handlers/
    └─ booking-events.ts
    └─ inventory-events.ts
    └─ payment-events.ts

└─ jobs/
  └─ inventory-reconciliation.ts
  └─ expired-holds-cleanup.ts
  └─ notification-scheduler.ts
  └─ stale-bookings-cleanup.ts

└─ utils/
  └─ id-generator.ts       # Booking ID generation
  └─ crypto.ts            # HMAC, hashing
  └─ date.ts              # Date utilities
  └─ logger.ts            # Logger setup

└─ package.json
└─ tsconfig.json

└─ src/                  # Frontend (React)
  └─ App.jsx
  └─ main.jsx
  └─ index.css
  └─ api/                # API client layer
  └─ components/         # Shared UI components
  └─ pages/              # Page components
  └─ hooks/              # Custom React hooks
  └─ context/            # React context providers
  └─ stores/             # State management
  └─ utils/              # Utility functions

└─ docs/
  └─ StayVeo_Engineering_Design_Document.md

└─ package.json

```

22. Deployment Strategy

22.1 MVP Deployment



22.2 CI/CD Pipeline (Future)



23. Monitoring & Observability

23.1 Key Metrics

Metric	Alert Threshold	Tool
API Response Time (p99)	> 500ms	Prometheus + Grafana
Error Rate	> 1%	Prometheus + PagerDuty
Booking Success Rate	< 95%	Custom dashboard
Payment Success Rate	< 98%	Razorpay dashboard + custom
Inventory Drift Count	> 0	Daily reconciliation cron
Notification Delivery Rate	< 90%	Custom dashboard
DLQ Size	> 10	Redis monitoring
Database Connection Pool	> 80% utilization	pg_stat
Redis Memory	> 80%	Redis INFO
CPU Usage	> 70% sustained	Infrastructure monitoring

23.2 Structured Logging

```
// Every log entry includes:
{
  "timestamp": "2026-07-20T12:38:47.123Z",
  "level": "info",
  "service": "booking",
  "traceId": "abc-123-def",      // distributed tracing
  "userId": "user-uuid",
  "action": "booking.create",
  "bookingId": "STV-2026-000042",
  "duration_ms": 145,
  "status": "success"
}
```

24. Future Improvements

24.1 Phase 2 Features

Feature	Priority	Complexity
Review & Rating System	HIGH	Medium
In-app Chat (Student ↔ Provider)	HIGH	High
Calendar Sync for Providers	MEDIUM	Medium
Smart Pricing (dynamic pricing based on demand)	MEDIUM	High
Virtual Tour (360° images)	LOW	Medium

FEATURE	PRIORITY	COMPLEXITY
Roommate Matching Algorithm	MEDIUM	High
Multi-language Support (i18n)	MEDIUM	Low
Provider Analytics Dashboard	HIGH	Medium
Student Verification (college ID)	HIGH	Low
Referral System	MEDIUM	Low

24.2 Technical Debt to Address

ITEM	EFFORT	PRIORITY
Migrate from HS256 to RS256 for JWT	1 day	Medium
Add comprehensive integration tests	2 weeks	High
Implement proper CQRS with read models	1 week	Medium
Add OpenTelemetry distributed tracing	3 days	Medium
Database connection pooling with PgBouncer	1 day	High
Implement GraphQL for mobile app (future)	2 weeks	Low
Add Terraform for infrastructure-as-code	1 week	Low

24.3 Microservice Extraction Plan

When traffic exceeds Phase 1 capacity:

```
STEP 1: Extract Notification Service
- Highest independence, lowest risk
- Communicate via Redis Streams / Kafka
- Own database for notification state

STEP 2: Extract Payment Service
- Critical path isolation
- Own database for payment records
- PCI compliance boundary

STEP 3: Extract Inventory Service
- Highest complexity
- Requires distributed transactions (saga)
- Own database for inventory state

STEP 4: Extract Search Service
- ElasticSearch as primary store
- CDC (Change Data Capture) from PostgreSQL
- Read-only service
```

Appendix A: Booking ID Sequence Setup

```
-- Run once during initial deployment
CREATE SEQUENCE IF NOT EXISTS booking_seq START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE IF NOT EXISTS invoice_seq START WITH 1 INCREMENT BY 1;
```

Appendix B: Cron Jobs

JOB	SCHEDULE	PURPOSE
Inventory Reconciliation	0 2 * * * (2 AM daily)	Detect and fix inventory drift
Expired Holds Cleanup	*/30 * * * * (every 30 min)	Release expired reservation holds
Notification Scheduler	0 * * * * (every hour)	Send scheduled notifications (reminders)
Stale Bookings Cleanup	0 3 * * * (3 AM daily)	Cancel bookings stuck in PAYMENT_PENDING > 1 hour
Daily Snapshots	0 0 * * * (midnight)	Take inventory snapshots for analytics
WhatsApp Inventory Check	0 20 * * * (8 PM daily)	Send WhatsApp inventory confirmation to providers

Appendix C: Environment Variables

```
# Application
NODE_ENV=production
PORT=3000
API_VERSION=v1

# Database
DATABASE_URL=postgresql://user:pass@host:5432/stayveo
DIRECT_URL=postgresql://user:pass@host:5432/stayveo

# Redis
REDIS_URL=redis://host:6379

# Supabase
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...

# Auth
JWT_SECRET=your-256-bit-secret
JWT_EXPIRY=15m
REFRESH_TOKEN_EXPIRY=7d

# Razorpay
RAZORPAY_KEY_ID=rzp_live_xxx
RAZORPAY_KEY_SECRET=your-secret
RAZORPAY_WEBHOOK_SECRET=your-webhook-secret

# Notification Providers
MSG91_AUTH_KEY=xxx
SENDGRID_API_KEY=xxx
FCM_SERVER_KEY=xxx
WHATSAPP_API_TOKEN=xxx

# Feature Flags
ENABLE_WHATSAPP_SYNC=true
ENABLE_OFFLINE_RECONCILIATION=true
RESERVATION_HOLD_HOURS=72
```

Document End

This Engineering Design Document is a living document. It should be updated as the system evolves and new requirements are identified. All architectural decisions should go through an RFC process before implementation.

Prepared by: CTO Office — StayVeo Engineering
Review Committee: Engineering Lead, Product Manager, Backend Lead

